

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-ab282ce950761f83f.jsonl`
- SHA-256: `53d5fe9f739b1c22385f4e8a8f11f8741504324ecf7bc6cdfcc4a13173c7ea68`
- Source modified: `2026-06-06T15:26:29+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d4359b2a-456`
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:23:31.664Z)

Produce a SINGLE Markdown document (the body of CODE_MAP.md). Audience: a future AI ramping up COLD on this codebase ? optimize for fast, unambiguous ingestion and DIRECT navigation, not prose. Dense notation over sentences, but EVERY anchor must be real.

Structure:

- Title + one-line purpose + a "Regenerate" line (rerun the build-code-map workflow) + a LEGEND defining the notation you use (suggested: @path = file, :: = symbol in file, -> = dataflow/call, \$ = data contract, ? = gotcha, ? = registry/plugin point).
- 1. PIPELINE ? the end-to-end runtime dataflow video-file -> highlight reel, AND the calibration/eval flow, as compact arrow chains with @path::symbol anchors. Show where each stage can be stopped (e.g. segments-only, no stitch).
- 2. SUBSYSTEMS ? one block per subsystem; under each, one dense line per important file: "@path Symbol(sig) ? role", with nested -> edges / \$ contracts / ? gotchas. Call out the registries (segmenter_registry, sport_registry, provider_registry, validator registry, annotation provider registry) and how a new plugin registers.
- 3. DATA CONTRACTS ? the canonical shapes in ONE place: trajectory CSV (Frame,Visibility,X,Y), candidate window dict (its keys), play-segment dict, golden labels CSV, det_raw.jsonl window record + manifest.json, eval Interval/Score/Match.
- 4. WASB WSL CONTRACT ? condense the verified facts provided.
- 5. CROSS-REPO ? condense the collector<->indexer facts provided.
- 6. GOTCHAS ? cross-cutting footguns (MD5 keying, Gemini free-tier dead path, fps/frame_width must be probed not assumed, WSL /mnt can't see Google Drive, ffmpeg not in WSL, GPU float nondeterminism, output/ gitignored, etc.).
- 7. RAMP PATHS ? a table "to do X -> open Y::symbol" for common change-tasks (tune windowing, add a segmenter, change stitching padding, add a sport, run calibration, run a video end-to-end, resume a crashed WASB pass).
- 8. TEST MAP ? which tests/test_*.py covers which subsystem.

Return ONLY the markdown body, no preamble.

=== SUBSYSTEM MAPS (JSON, from parallel readers) ===

```
[{"subsystem":"Orchestration & config","summary":"Top-level entrypoints that wire the badminton-highlight-indexer pipeline together. They all read config.json (JSON dict with sections sport/ai_provider/ai_model/validation/indexing/output/stitching), derive a video_id = MD5 of the video file, and drive the same backend components: validators (backend.validators.base.registry), pluggable segmenters (backend.indexer.segmenter_registry ? BasePlaySegmenter subclasses), the SQLite Database, and the ffmpeg VideoCompiler. backend/main.py is the unified orchestrator: a FastAPI server (uvicorn) plus a CLI (process / debug-clip / trim / setup). run_process_and_stitch.py is a hardcoded one-shot demo. run_eval.py / run_phase3_eval.py / run_regression.py are the evaluation harness CLIs (score DB-stored predictions vs ground-truth rally CSVs; no pipeline run). setup.py is the install/diagnostics bootstrapper. Cross-cutting contract: predictions must already be in output/<db_name> before any eval runs; video_id is the join key everywhere and MD5 chunk sizes differ per file (but all hash the whole file, so they agree).","files":[{"path":"backend/main.py","role":"Unified
```

```

orchestrator: FastAPI app + argparse CLI (server / process / debug-clip / trim / setup
launcher).", "symbols": [{"name": "app", "kind": "const", "sig": "FastAPI(title='Sports Highlight
Indexer API')", "role": "FastAPI app; CORS allow-all; mounts backend/static at /static; serves
index.html at
/."}, {"name": "db_instance", "kind": "const", "sig": "Optional[Database]", "role": "Module-global DB
handle set in main(); used by all /api endpoints."}, {"name": "global_config", "kind": "const", "sig":
"Optional[Dict[str, Any]]", "role": "Module-global parsed config.json (with output.output_dir
injected); served raw at
/api/config."}, {"name": "ProcessRequest", "kind": "class", "sig": "BaseModel{video_path:str,
player_pool:Optional[List[str]], max_frames:Optional[int],
segmenter:Optional[str]}", "role": "Body for POST
/api/process."}, {"name": "QueryRequest", "kind": "class", "sig": "BaseModel{video_id:str,
server/receiver:Optional[str], duration_min:Optional[float],
event_type:Optional[str]}", "role": "Body for POST /api/query ? builds the filters dict passed to
db.query_play_segments."}, {"name": "CompileRequest", "kind": "class", "sig": "BaseModel{video_id:str,
segment_ids:List[int], output_filename:str}", "role": "Body for POST
/api/compile."}, {"name": "process_video", "kind": "func", "sig": "(req:ProcessRequest)->dict @POST
/api/process", "role": "MD5?validate?add_video?segment; returns {video_id,status,message,results,p
lay_segments}."}, {"name": "query_play_segments", "kind": "func", "sig": "(req:QueryRequest)-
>{play_segments:[...]} @POST /api/query", "role": "Filtered segment lookup."}, {"name": "compile_hig
hlights", "kind": "func", "sig": "(req:CompileRequest)->{status,filepath} @POST
/api/compile", "role": "Selects segments by id, builds (start,end) pairs, runs
VideoCompiler."}, {"name": "get_config", "kind": "func", "sig": "()->global_config @GET
/api/config", "role": "Returns the live config
dict."}, {"name": "serve_index", "kind": "func", "sig": "()->HTMLResponse @GET /", "role": "Serves
backend/static/index.html or a
placeholder."}, {"name": "get_file_md5", "kind": "func", "sig": "(filepath:str)->str", "role": "Whole-
file MD5 in 64KB chunks = video_id."}, {"name": "load_config", "kind": "func", "sig": "(config_path='c
onfig.json')->Dict", "role": "Parse config.json or {} if
absent."}, {"name": "run_cli_diagnose_clip", "kind": "func", "sig": "(video_path:str,
timestamp:float)->None", "role": "--debug-clip: runs validators + measures avg OpenCV frame-diff
motion ±3s vs
indexing.motion_threshold."}, {"name": "main", "kind": "func", "sig": "()->None", "role": "argparse
dispatcher: --setup/--trim/--debug-clip/--video-path(CLI
process)/--server(uvicorn)."}], "contracts": ["Validator result object (from registry.run_all):
has .passed:bool, .validator_name:str, .message:str, .details, and .dict() ? stored via
db.add_video(...,[r.dict()]).", "Segment dict from query_play_segments: has keys id:int,
start_time:float, end_time:float (used by /api/compile and /api/query).", "db.get_video(video_id)
returns a record with 'filepath' key.", "Segmenter constructed as cls(db_instance,
global_config); process_video(video_id, video_path, player_pool=None,
max_frames=None)->segments.", "config.json sections read: indexing.default_segmenter (CLI default
'motion'), indexing.motion_threshold, output.db_name,
output.output_dir."], "edges": ["process_video ->
backend/validators/base.registry.run_all(video_path, global_config) (validation
gate)", "process_video/CLI -> backend/indexer/base.segmenter_registry.get(seg_name) +
.list_available() (pluggable segmenter lookup)", "process_video/CLI ->
backend/database.Database.add_video / query_play_segments (persist +
read)", "compile_highlights/trim ->
backend/stitcher/compiler.VideoCompiler.compile_highlights(filepath, time_pairs, name) (ffmpeg
stitch)", "main(--setup) -> subprocess run setup.py (delegated
diagnostics/install)", "main(--server) -> uvicorn.run(app, host=127.0.0.1,
port=args.port)"], "gotchas": ["db_instance/global_config are module globals only set inside
main() ? importing app for ASGI without running main() leaves them None and every endpoint
NPes.", "uvicorn binds host 127.0.0.1 hardcoded (localhost only) regardless of CLI; only --port
is configurable.", "CLI default_segmenter falls back to 'motion' here, but config.json ships
default_segmenter='gemini' and setup.py writes 'yolo_hybrid' ? three different defaults across
the subsystem.", "output.output_dir is injected into global_config at runtime (not in
config.json); db_name resolved relative to args.output_dir.", "run_cli_diagnose_clip does a lazy
`import cv2` inside the function; opens its own VideoCapture, samples ±3s (comment says
±5)."]}, {"path": "run_process_and_stitch.py", "role": "One-shot demo driver: hardcoded video path ?
segment (or cache) ? interactive stitch via VideoCompiler."}, {"symbols": [{"name": "get_file_md5", "k
ind": "func", "sig": "(filepath:str)->str", "role": "Whole-file MD5 in 1MB chunks with GB progress
print = video_id."}, {"name": "main", "kind": "func", "sig": "()->None", "role": "Load config, MD5
video, check DB cache, segment, filter by shot_count, compile
highlights."}], "contracts": ["Hardcoded paths: video_path / output_dir / output_filename are

```

```

literals in main() (must be edited to use).","Cache hit requires
db.get_video(video_id)['status']=='processed' AND query_play_segments returns >0.,"Segment
dict: start_time, end_time, metadata (dict OR JSON string ? parsed); metadata.shot_count:int
used by the min_shot_count filter.,"config sections: stitching.{use_cache,end_padding(1.0
default here),begin_padding(0.5),min_shot_count(0)}; indexing.default_segmenter (default
'yolo_hybrid')."},"edges":["main ->
backend/database.Database.{get_video,query_play_segments,clear_video_data,add_video} (cache +
persist)","main -> backend/indexer.segmenter_registry.get(seg_name) then
cls(db,config).process_video (run detector)","main ->
backend/stitcher/compiler.VideoCompiler.compile_highlights(video_path, time_pairs,
output_filename)","imports backend/validators/base.registry (imported but never used
here)","gotchas":["Interactive input() prompts (cache choice + padding overrides) ? blocks in
any non-TTY/automation context.,"Default end_padding/begin_padding here (1.0/0.5) differ from
config.json (0.5/0.5); these are fallback defaults only.,"default_segmenter fallback
'yolo_hybrid' differs from main.py's 'motion' and config.json's 'gemini'.","Configures root
logging.basicConfig at import time (only effective if it runs before other basicConfig
calls).","Skips OpenCV validation entirely ('Bypassing OpenCV validations') ? writes status
'processed' before segmenting.]],{"path":"run_eval.py","role":"Eval CLI: score a single video's
DB-stored rally predictions vs a ground-truth CSV (no pipeline run)."},"symbols":[{"name":"build_
parser","kind":"func","sig":"()->argparse.ArgumentParser","role":"Flags: --annotations,--video,
--video-id,--db,--stage{candidates,final,both},--detector,--iou,--pr-curve,--show-failures,--
json,--scaffold.},{name:"main","kind":"func","sig":"(argv=None)->int","role":"Resolve
video_id, load GT + DB, evaluate, print/JSON report. Exit 2 on arg/IO
errors.},{name:"get_file_md5","kind":"func","sig":"(filepath:str)->str","role":"Whole-file
MD5 in 8KB chunks (matches pipeline video_id).},{name:"_default_db_path","kind":"func","sig":
"()->str","role":"output/<config.output.db_name> (default
sports_indexer.db).},{name:"_EPILOG","kind":"const","sig":"str","role":"Help text: harness
scores DB predictions, does NOT run the pipeline.}],{"contracts":["Ground truth from
load_annotations(path): list of intervals (start,end per CSV row).","stage='both' expands to
list(STAGES)=('candidates','final').","evaluate(db, video_id, gt, stages, iou_threshold,
detector, with_pr_curve)->EvalReport; serialized by report_to_dict.,"config.json read only for
output.db_name when --db absent.],"edges":["main -> backend/eval/annotations.load_annotations /
write_scaffold (--scaffold writes empty CSV)","main -> backend/eval/harness.evaluate /
format_report_text / report_to_dict / STAGES (scoring + reporting)","main ->
backend/database.Database(db_path) (read predictions)","gotchas":["Pure scorer: errors with
'database not found' / zero predictions if the video was never processed first.,"Exit code 2 =
arg/usage errors; success always returns 0 (no regression gating here ? that's
run_regression.py).","--video recomputes MD5 every run on large files; prefer --video-
id.]],{"path":"run_phase3_eval.py","role":"Phase-3 batch driver: walk a collector
manifest.json, optionally segment each video, score vs per-video GT CSV, aggregate.},"symbols":[{"name":"build_parser","kind":"func","sig":"()->ArgumentParser","role":"Flags: --manifest(req),
--videos-root,--segmenter(default motion),--stage{...,auto},--detector,--iou,--only(repeatable),
--limit,--max-frames,--score-only,--reprocess,--db,--json,--show-
failures.},{name:"main","kind":"func","sig":"(argv=None)->int","role":"Per-video loop:
resolve path, MD5, segment-if-needed, evaluate, accumulate; print aggregate +
skipped.},{name:"load_config","kind":"func","sig":"(path='config.json')->dict","role":"Parse
config or {}.},{name:"get_file_md5","kind":"func","sig":"(filepath:str)->str","role":"Whole-
file MD5 in 64KB
chunks.},{name:"logger","kind":"const","sig":"logging.getLogger(__name__)","role":"Module
logger (basicConfig set in main).}],{"contracts":["manifest.json shape: {eval_sets:[
{video_id:str, local_path:str, csv:str}, ... ]}; csv resolved relative to manifest dir;
local_path resolved against videos_root.,"videos_root default = two dirs above manifest (the
collector repo root).","stages = batch.default_stages_for(segmenter, stage) (resolves
'auto').","per_video entries = report_to_dict(report) + {'collector_video_id': manifest
video_id}.","JSON out: {segmenter,iou,stages,aggregate,per_video,skipped};
skipped=[(video_id,reason),...].","NOTE: video_id used for DB ops = MD5 of the file
(get_file_md5), NOT the manifest's video_id (which is a label only)."],"edges":["main ->
backend/eval/batch.{resolve_video_path,default_stages_for,aggregate,format_aggregate} (path
resolution + corpus rollup)","main -> backend/eval/annotations.load_annotations (per-video
GT)","main -> backend/eval/harness.evaluate/format_report_text/report_to_dict (scoring)","main
-> backend/indexer.segmenter_registry.get(args.segmenter) + .list_available() (segment unless
--score-only)","main ->
backend/database.Database.{query_play_segments,query_candidate_segments,add_video} (skip-if-
present + persist)","gotchas":["load_dotenv() at import so GEMINI_API_KEY is available;
yolo_hybrid/gemini segmenters cost API calls ? default 'motion' is free/CPU.,"Already-present

```

check uses query_play_segments OR query_candidate_segments; --reprocess forces re-segmentation.", "manifest 'video_id' is a human label; the actual DB key is the file MD5 ? mixing them up breaks lookups.", "--max-frames passed straight into segmenter.process_video for fast validation."}], {"path": "run_regression.py", "role": "Regression CLI: score DB predictions via an annotation-provider tier vs a snapshotted baseline; exit 1 on regression (CI gate).", "symbols": [{"name": "build_parser", "kind": "func", "sig": "()->ArgumentParser", "role": "Flags: --video-id(req), --tier(default file), --annotations(video_ref), --stage{candidates,final}, --detector, --iou, --tolerance, --db, --baseline, --update-baseline, --json."}, {"name": "main", "kind": "func", "sig": "(argv=None)->int", "role": "regress_with_provider?print?optionally save baseline/JSON; return 1 if regressed else 0 (2 on missing DB)."}, {"name": "_default_db_path", "kind": "func", "sig": "()->str", "role": "output/<config.output.db_name>."}], "contracts": ["RegressionResult: .video_id, .stage, .precision, .recall, .f1, .regressed:bool, .as_dict() (JSON dump).", "Baseline JSON written via save_baseline(video_id, stage, precision, recall, f1, path).", "Defaults pulled from regression module: DEFAULT_TOLERANCE=0.05, DEFAULT_BASELINE_PATH=output/regression_baseline.json.", "--annotations is the provider video_ref (CSV path for the 'file' tier)."], "edges": ["main -> backend/eval/regression.regress_with_provider(db, video_id, video_ref, tier, stage, iou_threshold, detector, tolerance, baseline_path) (core compare)", "main -> backend/eval/regression.{format_result, save_baseline, DEFAULT_TOLERANCE, DEFAULT_BASELINE_PATH} (report + snapshot)", "build_parser -> backend/annotations.annotation_registry.list_available() (tier choices in --help)", "main -> backend/database.Database(db_path)", "gotchas": ["Exit codes are semantic: 0=ok, 1=REGRESSION DETECTED (gates CI), 2=missing DB ? callers must distinguish 1 vs 2.", "--update-baseline runs AFTER the comparison print, so it snapshots current numbers even on a regressing run.", "tier 'file' uses --annotations as a CSV path; other registered tiers (e.g. 'auto') interpret video_ref differently."], {"path": "config.json", "role": "Single runtime config consumed by every entrypoint (sport/AI provider, validation thresholds, segmenter/indexing params, output, stitching).", "symbols": [{"name": "validation", "kind": "const", "sig": "{min_resolution_width/height, min_fps, min_blur_threshold, max_scene_cuts_per_minute, relevance{court_green_lower/upper(HSV), court_pixels_min_ratio}}", "role": "Thresholds for backend/validators/base.registry.run_all."}, {"name": "indexing", "kind": "const", "sig": "{default_segmenter: 'gemini', use_gpu, motion_threshold, min_segment_duration, dead_time_merge_gap, chunk_duration, detector_model, detector_score_threshold, yolo_*, min_action_window_duration, log/store_yolo_candidates, ai_handoff_padding, ai_max_workers, tracknet{...}}", "role": "All segmenter/detector tunables."}, {"name": "indexing.tracknet", "kind": "const", "sig": "{wsl_distro: 'Ubuntu', repo_dir, conda_sh, conda_env, weights_path, queue_length, stage_in_wsl, wsl_stage_dir, timeout_sec, velocity_threshold}", "role": "WSL2/conda config for the self-hosted TrackNet/WASB shuttle model."}, {"name": "output", "kind": "const", "sig": "{default_highlights_name: 'highlights.mp4', db_name: 'sports_indexer.db'}", "role": "Output filenames; db_name resolved under output/."}, {"name": "stitching", "kind": "const", "sig": "{begin_padding: 0.5, end_padding: 0.5, use_cache: false, min_shot_count: 1}", "role": "VideoCompiler padding + cache + shot-count filter."}, {"name": "(root)", "kind": "const", "sig": "{sport: 'badminton', ai_provider: 'gemini', ai_model: 'gemini-2.5-flash'}", "role": "Top-level sport + LLM provider/model selection."}], "contracts": ["court_green_lower/upper are 3-element HSV triples; consumed by the relevance validator.", "indexing.default_segmenter must be a key registered in segmenter_registry; output.db_name joins with output/ to form the DB path.", "All entrypoints load this via json.load and tolerate absence (fall back to {})."], "edges": ["consumed-by run_process_and_stitch.py/run_eval.py/run_phase3_eval.py/run_regression.py/backend.main (json.load)", "indexing.tracknet -> backend WASB/TrackNet substrate (WSL2 conda invocation params)", "gotchas": ["Live config ships default_segmenter='gemini' (costs API) while setup.py's generated default is 'yolo_hybrid' and main.py's code fallback is 'motion' ? easy to get a different segmenter than expected.", "stitching.use_cache=false here vs run_process_and_stitch default True ? only matters when key is absent.", "min_blur_threshold=20.0 here but setup.py's template writes 100.0; relevance{} block exists here but not in setup.py's template.", "tracknet.weights_path is empty by default ? WASB inference needs it populated; timeout_sec=0 means no timeout."], {"path": "setup.py", "role": "Bootstrapper: write default config.json, run FFMpeg/PyTorch/CUDA diagnostics, pip-install requirements with GPU-aware wheel index.", "symbols": [{"name": "init_default_config", "kind": "func", "sig": "()->None", "role": "Write a default config.json (skips if present)."}, {"name": "run_diagnostics", "kind": "func", "sig": "()->bool", "role": "Check Python>=3.8, ffmpeg/ffprobe on PATH, torch+CUDA availability; returns ffmpeg&ffprobe ok."}, {"name": "run_pip_install", "kind": "func", "sig": "()->bool", "role": "pip install --user -r requirements.txt with cu124 or cpu extra-index-url based on nvidia-smi."}, {"name": "check_command", "kind": "func", "sig": "(cmd:str)->bool", "role": "shutil.which presence check."}], "contracts": ["Generated config.json mirrors config.json structure but: default_segmenter='yolo_hybrid', min_blur_threshold=100.0, NO validation.relevance block, NO indexing.tracknet block.", "Invoked by backend/main.py --setup via subprocess([sys.executable,

```
'setup.py']))."],"edges":["init_default_config -> writes config.json (consumed by all
orchestration entrypoints)","invoked-by backend/main.py main(--setup)
(subprocess)"],"gotchas":["GPU detection keys off `nvidia-smi` on PATH ? selects cu124 vs cpu
PyTorch wheels; a CUDA box without nvidia-smi gets CPU wheels.","pip install uses --user (can
clash with venvs).","Its generated config diverges from the checked-in config.json (segmenter,
blur threshold, missing relevance/tracknet sections) ? never run on an existing repo expecting
parity (it no-ops if config.json exists)."]]],{"subsystem":"Segmenters
(indexer)","summary":"backend/indexer/ holds the pluggable play-segmenter subsystem. base.py
defines the BasePlaySegmenter ABC + a module-level singleton SegmenterRegistry; each concrete
segmenter self-registers at import time by calling segmenter_registry.register(...) at end-of-
module, and __init__.py imports all of them so the registry is populated as a side-effect. Every
segmenter implements process_video(video_id, video_path, player_pool=None, max_frames=None) ->
List[segment dict] and writes to the SQLite Database (db.add_play_segment /
add_candidate_segment / link_candidate_to_segment / add_event). Three \"hybrid\" segmenters
(yolo_hybrid, tracknet_hybrid, wasb_hybrid) share an identical 4-phase shape: Phase 2 generates
candidate \"action windows\" from local motion/trajectory, Phase 3 hands padded clips to an AI
provider (provider_registry) for semantic boundaries via a ThreadPoolExecutor, Phase 4 writes
play_segments and links them back to candidates. tracknet/wasb differ only in the Phase-2
substrate (WSL2 shuttle-trajectory runners) ? they are near-verbatim copies. gemini.py is the
pure-AI segmenter (no local Phase 2; chunk-or-single-file + heavy post-hoc validation).
motion.py is the legacy pixel-diff CV baseline that fabricates random metadata/events. Cross-
module config lives under config[\"indexing\"]; sport prompt text comes from sport_registry.get(
sport).get_prompt(chunk_duration=...)."],"files":[{"path":"backend/indexer/base.py","role":"Abstr
act base class + the global segmenter registry that every concrete segmenter self-registers
into."},"symbols":[{"name":"BasePlaySegmenter","kind":"class","sig":"ABC; __init__(self, db:
Database, config: Dict[str,Any])","role":"Base for all segmenters; stores self.db + self.config;
subclasses implement name/description/process_video."},{name":"BasePlaySegmenter.name","kind":"
func","sig":"@property @abstractmethod -> str","role":"Unique registry key for the
segmenter."},{name":"BasePlaySegmenter.description","kind":"func","sig":"@property
@abstractmethod -> str","role":"Human-readable blurb (surfaced by registry.list_available)."},{
name":"BasePlaySegmenter.process_video","kind":"func","sig":"@abstractmethod (video_id: str,
video_path: str, player_pool: List[str]=None, max_frames: int=None) ->
List[Dict[str,Any]]","role":"Core entrypoint: detect segments, populate DB, return segment dicts
."},{name":"SegmenterRegistry","kind":"class","sig":"register(cls)/get(name)-
>Optional[Type]/list_available()->Dict[str,str]","role":"Name->class map; resolves
name/description by instantiating
cls(None,{})."},{"name":"segmenter_registry","kind":"registry","sig":"SegmenterRegistry() module
singleton","role":"The one shared registry instance all segmenters register into and consumers
look up."}],contracts":["BasePlaySegmenter.__init__(db, config): config is a nested dict;
segmenters read config['sport'], config['ai_provider'], config['ai_model'], config['indexing']
(and config['stitching'] in gemini).","SegmenterRegistry.register instantiates
segmenter_cls(None, {}) (db=None, config={}) JUST to read the .name property ? so
name/description must NOT touch self.db or self.config or registration raises
ValueError."],"process_video return contract: list of dicts with keys id, start_time, end_time,
duration, metadata (motion.py also adds server/receiver)."],"edges":["BasePlaySegmenter.__init__
-> backend/database.Database (type of self.db; concrete segmenters call its add_*
methods)","register/list_available -> segmenter_cls(None, {}) (constructs with null db to read
name/description)"],"gotchas":["register() builds a throwaway instance with db=None and
config={}: any segmenter whose name/description property dereferences self.db/self.config would
break registration. All current segmenters keep these pure."],"Registration is an IMPORT SIDE
EFFECT: a segmenter only exists in the registry if its module is imported. __init__.py imports
all of them; importing base alone yields an empty registry."],"config={} default means every
indexing key access in subclasses must use .get with a default ? there is no schema/validation
here."}],{"path":"backend/indexer/__init__.py","role":"Package aggregator: imports every
segmenter module so they self-register, and re-exports the public
names."},"symbols":[{"name":"__all__","kind":"const","sig":"list[str]","role":"Public API:
segmenter_registry, BasePlaySegmenter, and all 5 concrete segmenter
classes."}],contracts":["Importing backend.indexer triggers import of motion, gemini,
yolo_hybrid, tracknet_hybrid, wasb_hybrid ? populating segmenter_registry with keys: motion,
gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid."],"edges":["imports ->
base.segmenter_registry, base.BasePlaySegmenter","imports -> motion.MotionPlaySegmenter /
gemini.GeminiPlaySegmenter / yolo_hybrid.HybridYoloSegmenter /
tracknet_hybrid.TrackNetHybridSegmenter / wasb_hybrid.WasbHybridSegmenter (each self-registers
on import)"],"gotchas":["This file is the de-facto registration manifest: adding a new segmenter
requires importing it here or it will never appear in segmenter_registry."],"Import order here is
```

```

also registry-insertion order; importing wasb_hybrid/tracknet_hybrid pulls in their WSL runner
modules (heavier import surface)."}],{"path":"backend/indexer/wasb_hybrid.py","role":"Hybrid
segmenter: WASB (MIT) shuttle-trajectory via WSL2 for candidate rallies + AI provider for
semantic boundaries. Live shuttle
substrate."}, {"symbols":[{"name":"WasbHybridSegmenter","kind":"class","sig":"BasePlaySegmenter;
name='wasb_hybrid'", "role":"4-phase pipeline; near-identical twin of TrackNetHybridSegmenter."},
{"name":"WasbHybridSegmenter.process_video","kind":"func","sig":"(video_id, video_path,
player_pool=None, max_frames=None) -> List[Dict]","role":"Healthcheck WSL -> run_predict ->
parse_trajectory_csv -> trajectory_to_action_windows -> store candidates -> AI handoff ->
DB."}, {"name":"WasbHybridSegmenter._probe_fps_width","kind":"func","sig":"@staticmethod
(video_path) -> (fps, width); fallback (30.0, 1280.0)","role":"cv2 probe of source fps + frame
width for trajectory->window conversion."}, {"name":"_fmt_ts","kind":"func","sig":"(seconds:
float) -> 'HH:MM:SS'","role":"Timestamp formatter for per-rally candidate log
lines."}], "contracts":["candidate window dict (from runner.trajectory_to_action_windows): keys
start, end, active_frame_count, peak_velocity, mean_velocity, track_id_count,
chunk_index.", "db.add_candidate_segment(video_id, detector='wasb_hybrid', start_time, end_time,
chunk_index, confidence=min(1.0, peak_velocity),
stats={peak_velocity, mean_velocity, active_frame_count, track_id_count}, detection_params={detecto
r, velocity_thresh, merge_gap, min_action_window_duration, weights_path}).", "AI segment dict
consumed: start_time, end_time (offset by w_start), shuttle_exchange_count?,
shot_count_confidence?; internal key _candidate_id stamped per segment.", "Phase-4 metadata
written: {shot_count, shot_count_confidence='Unknown', detector=f'wasb-
hybrid-{model_name}'}.", "Config read: config['indexing'].wasb.velocity_threshold (0.02),
dead_time_merge_gap (2.0), min_action_window_duration (2.0), ai_handoff_padding (2.0),
ai_max_workers (5), log_yolo_candidates (True), store_yolo_candidates
(True)."], "edges":["process_video -> WasbConfig.from_indexing_cfg(idx_cfg) / WasbRunner
(backend/detectors/wasb_runner.py: healthcheck, run_predict, parse_trajectory_csv,
trajectory_to_action_windows)", "process_video ->
sport_registry.get(sport).get_prompt(chunk_duration=) (backend/sports)", "process_video ->
provider_registry.get(provider_name, api_key, model_name).analyze_video(path, prompt,
chunk_duration=, label=) (backend/providers)", "process_video ->
backend/utils/video.get_video_duration + extract_video_chunk(reencode=True for handoff
clips)", "process_video -> db.add_candidate_segment / add_play_segment /
link_candidate_to_segment (backend/database)"], "gotchas":["Requires a working WSL2 WASB env:
runner.healthcheck() gates the whole run; if not ok it logs and returns [] (silent no-op for
callers).", "Requires {PROVIDER}_API_KEY env var (e.g. GEMINI_API_KEY); missing key -> returns
[].", "Reuses YOLO-named config keys (log_yolo_candidates/store_yolo_candidates) and the wasb-
specific velocity_threshold lives under indexing.wasb, but merge_gap/min_window come from the
shared top-level indexing keys.", "Processes the WHOLE video in one WASB pass (chunk_index hard-
coded 0, chunk_start 0.0) ? no pre-chunking, unlike yolo_hybrid.", "shot_count fallback differs
subtly from tracknet: here it always int()-casts shuttle_exchange_count and only falls back on
ValueError/TypeError (a None value would raise TypeError and fall back to
max(3, int(dur/0.8))).", "Output dirs are hard-coded relative paths ('output/wasb',
'output/chunks_{provider}_handoff') ? cwd-sensitive; rmdir best-
effort."}], {"path":"backend/indexer/tracknet_hybrid.py","role":"Hybrid segmenter: TrackNetV4
shuttle-trajectory via WSL2 for candidate rallies + AI provider for boundaries. Near-verbatim
twin of wasb_hybrid."}, {"symbols":[{"name":"TrackNetHybridSegmenter","kind":"class","sig":"BasePla
ySegmenter; name='tracknet_hybrid'", "role":"Same 4-phase pipeline as WasbHybridSegmenter,
swapping the
runner."}, {"name":"TrackNetHybridSegmenter.process_video","kind":"func","sig":"(video_id,
video_path, player_pool=None, max_frames=None) -> List[Dict]","role":"Healthcheck -> run_predict
-> parse_trajectory_csv -> trajectory_to_action_windows -> store -> AI handoff ->
DB."}, {"name":"TrackNetHybridSegmenter._probe_fps_width","kind":"func","sig":"@staticmethod
(video_path) -> (fps, width); fallback (30.0, 1280.0)","role":"cv2 fps+width
probe."}, {"name":"_fmt_ts","kind":"func","sig":"(seconds) -> 'HH:MM:SS'","role":"Candidate-log
timestamp formatter."}], "contracts":["candidate window dict identical to wasb: start, end,
active_frame_count, peak_velocity, mean_velocity, track_id_count,
chunk_index.", "db.add_candidate_segment(..., detector='tracknet_hybrid', ..., detection_params
adds queue_length=tn_cfg.queue_length AND weights_path).", "Phase-4 metadata: {shot_count,
shot_count_confidence='Unknown', detector=f'tracknet-hybrid-{model_name}'}.", "Config:
indexing.tracknet.velocity_threshold (0.02) plus shared dead_time_merge_gap/min_action_window_du
ration/ai_handoff_padding/ai_max_workers/log_yolo_candidates/store_yolo_candidates."], "edges":["
process_video -> TrackNetConfig.from_indexing_cfg / TrackNetRunner
(backend/detectors/tracknet_runner.py: healthcheck, run_predict, parse_trajectory_csv,
trajectory_to_action_windows; exposes weights_path + queue_length)", "process_video ->

```

```

sport_registry.get(...).get_prompt / provider_registry.get(...).analyze_video /
backend/utils/video /
db.add_candidate_segment+add_play_segment+link_candidate_to_segment"],"gotchas":["WSL2-only like
wasb; healthcheck gates the run, missing API key -> [].", "whole-video single pass (TrackNet
carries its own frame buffering); chunk_index=0, chunk_start=0.0 ? comment explicitly notes no
pre-chunking.", "detection_params includes queue_length (TrackNet-specific) which wasb_hybrid
lacks ? the only real structural diff in Phase 2 metadata.", "This file and wasb_hybrid.py are
copy-paste twins: any fix to the AI-handoff / Phase-4 logic must be applied to BOTH (and
arguably yolo_hybrid).", "shot_count fallback here checks `if shot_count is None` first (slightly
different control flow than wasb's try/except-only path, but same
outcome)."], {"path": "backend/indexer/yolo_hybrid.py", "role": "Hybrid segmenter: torchvision
person-detection + supervision ByteTrack (Stage 1, local, license-clean) feeding an AI provider
for boundaries. Registry key kept 'yolo_hybrid' for back-compat though no YOLO
remains.", "symbols": [{"name": "HybridYoloSegmenter", "kind": "class", "sig": "BasePlaySegmenter;
__init__ adds self.model/self.device; name='yolo_hybrid'", "role": "Pre-chunks video, runs
detection+tracking per chunk, merges overlapping candidates, then AI handoff +
DB."}, {"name": "HybridYoloSegmenter._lazy_load_model", "kind": "func", "sig": "()" ->
None, "role": "Lazy torchvision detector load on cuda/cpu (so module imports without torchvision;
tests pre-set
self.model)."}, {"name": "HybridYoloSegmenter._make_tracker", "kind": "func", "sig": "(fps: float) ->
sv.ByteTrack", "role": "Fresh per-chunk ByteTrack (track IDs are chunk-
local)."}, {"name": "HybridYoloSegmenter._detect_and_track", "kind": "func", "sig": "(frame, tracker,
score_thresh) -> List[(track_id:int, bbox:(x1,y1,x2,y2))]", "role": "torchvision detect (person
class) -> ByteTrack associate -> (id,bbox) px tuples."}, {"name": "HybridYoloSegmenter._extract_ac
tion_windows_from_chunk", "kind": "func", "sig": "(chunk_path, chunk_start, velocity_thresh,
merge_gap, frame_skip=3, tracker_config='bytetrack.yaml', chunk_index=None) ->
List[Dict]", "role": "Per-chunk: velocity-based active-frame grouping into windows with motion
stats."}, {"name": "HybridYoloSegmenter.process_video", "kind": "func", "sig": "(video_id, video_path,
player_pool=None, max_frames=None) -> List[Dict]", "role": "Full pipeline: chunk -> windows ->
dedup-merge -> store candidates -> AI handoff ->
DB."}, {"name": "_PERSON_CLASS_ID", "kind": "const", "sig": "= 1", "role": "COCO person class for
torchvision (NB: ultralytics used 0 ? named off-by-one
gotcha)."}, {"name": "_DETECTOR_FACTORIES", "kind": "registry", "sig": "dict[str,str] of torchvision
detection builder names", "role": "Selectable BSD detectors via
indexing.detector_model."}, {"name": "_DEFAULT_DETECTOR", "kind": "const", "sig": "=
'fasterrcnn_resnet50_fpn'", "role": "Fallback detector when config value
unknown/missing."}, {"name": "_fmt_ts", "kind": "func", "sig": "(seconds) ->
'HH:MM:SS'", "role": "Candidate-log timestamp formatter."}], "contracts": ["candidate window dict
(same shape consumed by Phase 3/4 as the trajectory hybrids): start, end, active_frame_count,
peak_velocity, mean_velocity, track_id_count, chunk_index.", "_detect_and_track return:
List[Tuple[int track_id, Tuple[float x1,y1,x2,y2] pixel bbox]].", "db.add_candidate_segment(...,
detector='yolo_hybrid', confidence=min(1.0, peak_velocity),
stats={peak_velocity, mean_velocity, active_frame_count, track_id_count}, detection_params={velocit
y_thresh, merge_gap, frame_skip, tracker, detector_model, detector_score_threshold, min_action_window
duration}).", "Phase-4 metadata: {shot_count, shot_count_confidence='Unknown', detector='f' yolo-
hybrid-{model_name}'}.", "Config: indexing.{use_gpu(True), detector_model,
detector_score_threshold(0.5), chunk_duration(300), chunk_overlap(30),
yolo_velocity_threshold(0.15), dead_time_merge_gap(2.0), yolo_frame_skip(3), yolo_tracker,
ai_handoff_padding|gemini_handoff_padding(2.0), ai_max_workers|gemini_max_workers(5),
log_yolo_candidates, store_yolo_candidates, yolo_prefetch_workers(2),
min_action_window_duration(2.0)}.", "edges": ["_lazy_load_model -> torchvision.models.detection
(lazy import; builder(weights='DEFAULT'))", "_make_tracker / _detect_and_track -> supervision as
sv (ByteTrack, Detections)", "_detect_and_track velocity ->
backend/utils/geometry.get_velocity_normalised (per-frame normalised
displacement)", "process_video -> backend/utils/video.get_video_duration + extract_video_chunk
(chunks copy-extracted; handoff clips reencode=True)", "process_video ->
sport_registry.get(...).get_prompt / provider_registry.get(...).analyze_video / db.add_candidate
_segment+add_play_segment+link_candidate_to_segment"],"gotchas":["_PERSON_CLASS_ID=1 is the
COCO/torchvision convention; ultralytics used 0 ? wrong value silently yields zero
detections.", "GPU inference is single-threaded by design (one GPU); only ffmpeg chunk EXTRACTION
is parallelized via yolo_prefetch_workers prefetch threads ? inference loop still runs chunks in
submission order.", "frame_skip (default 3) is applied via frame_idx % frame_skip; velocity is
computed at effective_fps = fps/frame_skip ? changing frame_skip rescales velocities, so
velocity_threshold is coupled to it.", "track_history is pruned to current_ids each frame to
avoid false velocity spikes on tracker ID reuse; trackers are per-chunk so cross-chunk identity

```

```

is NOT preserved (windows merged by TIME, not ID).","Overlapping windows across chunk_overlap
are merged with frame-weighted mean_velocity in _merge_stats; merge keeps a['chunk_index'] and
earliest start.","imports torch at module top (unlike torchvision/supervision which are lazy) ?
module import needs torch present.","tracker_config / yolo_tracker params are dead (retained for
back-compat); ByteTrack ignores the yaml.","extract_video_chunk uses stream-copy for chunks
(keyframe drift acceptable) but reencode=True for short handoff
clips."},{ "path":"backend/indexer/motion.py","role":"Legacy pure-CV baseline: pixel frame-
difference motion heuristics -> segments, with RANDOMLY FABRICATED metadata + serve/smash/foul
events. Demo/placeholder, not production-
truthful."},{ "name":"MotionPlaySegmenter","kind":"class","sig":"BasePlaySegmenter;
name='motion',"role":"absdiff motion-ratio thresholding -> merged/filtered segments -> DB with
synthetic events."},{ "name":"MotionPlaySegmenter.process_video","kind":"func","sig":"(video_id,
video_path, player_pool=None, max_frames=None) -> List[Dict]","role":"Subsample @5fps, smoothed
motion-ratio boundaries, merge gaps, filter short, write segments+random
events."}], "contracts":["Return dicts include extra keys vs the hybrids: id, start_time,
end_time, duration, server, receiver, metadata.","metadata fabricated: {shot_count:
randint(4,25), intensity: 'high' if dur>12 else 'medium', avg_speed_kmh:
random}.","db.add_play_segment(video_id, start, end, server, receiver, metadata) ? only
segmenter that actually passes server/receiver (random.sample from
player_pool).","db.add_event(segment_id, timestamp, event_type in {serve,smash,foul,winner},
player, details={...}) ? synthetic per-segment events.","Config:
indexing.{motion_threshold(0.08), min_segment_duration(3.0),
dead_time_merge_gap(2.0)}.","Default player_pool when falsy:
['Avi','Suresh','Ramesh','Deepak']."],"edges":["process_video ->
backend/database.Database.add_play_segment + add_event (only segmenter using add_event)","self-
registers -> base.segmenter_registry.register(MotionPlaySegmenter)"],"gotchas":["ALL rich
metadata and every serve/smash/foul/winner event is RANDOM (random.sample/randint/uniform) ? it
is fabricated, not detected. Do not treat as ground truth (conflicts with the project's no-
fabrication working agreement for real data).","No AI provider, no candidate-segment persistence
(does not call add_candidate_segment), and no API key needed ? fully local/offline.","Uses
print() not the logging module (the hybrids/gemini use logger); progress lines go to
stdout.","Uses cap.grab() to skip intermediate frames at analysis_fps=5.0 for speed; frame_idx
accounting interleaves grab + read.","max_frames is honored here (halts parsing) ? the
hybrid/gemini segmenters accept the param but largely ignore
it."}],{"path":"backend/indexer/gemini.py","role":"Pure-AI segmenter: sends whole video (or
overlapping chunks) to an AI provider, then runs heavy post-hoc validation (parse/clamp/dedup)
before DB write. No local Phase-2
detection."},{ "name":"GeminiPlaySegmenter","kind":"class","sig":"BasePlaySegmenter;
name='gemini',"role":"Optional ffmpeg debug-trim, chunk-or-single-file provider calls,
validate, write
segments."},{ "name":"GeminiPlaySegmenter.process_video","kind":"func","sig":"(video_id,
video_path, player_pool=None, max_frames=None) -> List[Dict]","role":"Trim(debug) ->
chunk/single provider.analyze_video -> offset timestamps -> validate ->
DB."},{ "name":"parse_time (nested)","kind":"func","sig":"(val) -> float seconds","role":"Robust
timestamp parse: float/int pass-through, numeric str, and MM:SS / HH:MM:SS (warns), else
0.0."}], "contracts":["AI segment dict consumed: start_time, end_time (offset by chunk_start in
chunked mode), shuttle_exchange_count?, shot_count_confidence?.","provider.analyze_video(path,
prompt, chunk_duration=, label=) -> (segments: list, metrics: dict). metrics keys read:
upload_time, process_time, gen_time (+ file_size_mb/segments_found stamped locally).","Phase-4
metadata: {shot_count, shot_count_confidence='Unknown', detector=model_name} (note: detector is
the bare model_name, NOT a 'hybrid-' prefix).","Validation rules: drop start>=end; drop
start>=video_duration; clamp end<video_duration; drop dur<min_segment_duration; warn (keep)
dur>max_segment_duration=90.0; resolve overlaps by keeping the longer segment.","Config:
indexing.{debug_duration, chunk_duration(300), chunk_overlap(30), min_segment_duration(3.0)};
stitching.{begin_padding(0.5), end_padding(0.5)} (used only for a duration-estimate
log)."],"edges":["process_video -> sport_registry.get(sport).get_prompt(chunk_duration=)
(backend/sports)","process_video -> provider_registry.get(provider_name, api_key,
model_name).analyze_video(...) (backend/providers)","process_video ->
backend/utils/video.get_video_duration + extract_video_chunk (chunked mode)","process_video ->
subprocess ffmpeg -c copy (debug_duration trim only)","process_video ->
backend/database.Database.add_play_segment (server/receiver passed as
None)"],"gotchas":["chunking decision: only chunks when video_duration > chunk_duration (else
single-file). Chunked path uses as_completed (order not preserved) then dedup sorts by start
anyway.","Chunk workers hard-coded max_workers=5 (does NOT read ai_max_workers, unlike the
hybrid segmenters).","debug_duration trims via ffmpeg -c copy to first N seconds for fast

```

```

iteration ? a footgun if left set in prod config (silently truncates analysis).", "parse_time
converts colon timestamps (MM:SS/HH:MM:SS) even though the prompt asks for decimal seconds ?
emits a warning; malformed -> 0.0.", "max_segment_duration=90.0 is hard-coded (not configurable);
long segments are kept with a warning, not dropped.", "Does not persist candidate_segments and
does not link_candidate_to_segment (no Phase-2 candidates exist here).", "Requires
{PROVIDER}_API_KEY env var; missing -> returns []. player_pool/max_frames are accepted but
unused."]]], {"subsystem": "Detectors & windowing", "summary": "Two shuttle-trajectory detector
backends (TrackNetV4 and WASB), both quarantined inside WSL2 so the main Windows Python env
never imports TF/Torch/CUDA. Windows-side adapters (tracknet_runner, wasb_runner) shell out via
`wsl bash -c \"source conda.sh && ...\"`, stage the video into the WSL fs (avoiding slow /mnt/c
reads), run the model, and read back a `Frame,Visibility,X,Y` trajectory CSV through the /mnt
view. The model-AGNOSTIC brain ? `parse_trajectory_csv` (CSV -> List[TrajectoryPoint]) and
`trajectory_to_action_windows` (velocity-thresholded candidate-rally windows) ? lives as
@staticmethods on TrackNetRunner and is re-exported verbatim by WasbRunner, so both detectors
emit the identical action-window dict shape that downstream AI-handoff/DB phases consume
(mirrors HybridYoloSegmenter). wasb_infer.py is the actual GPU inference wrapper that runs
INSIDE WSL (imports WASB's own detectors/trackers/dataloaders); its standout feature is a
reboot-resumable detector cache (det_raw.jsonl + atomic manifest.json) that checkpoints the
expensive sliding-window GPU pass so a crash loses at most `flush_every` windows, while the
cheap stateful CPU tracker pass is always re-run from the full ordered
cache.", "files": [{"path": "backend/detectors/tracknet_runner.py", "role": "Windows-side adapter
that runs TrackNetV4 predict.py in a WSL conda env and hosts the shared, model-agnostic CSV-
parsing + action-windowing logic reused by every detector
backend.", "symbols": [{"name": "TrackNetConfig", "kind": "dataclass", "sig": "distro='Ubuntu',
repo_dir='~/models/TrackNetV4', conda_sh, conda_env='TrackNetV4', weights_path='',
queue_length=5, stage_in_wsl=True, wsl_stage_dir='~/clips', timeout_sec=0", "role": "Resolved
settings for a TrackNet WSL
run."}], {"name": "TrackNetConfig.from_indexing_cfg", "kind": "func", "sig": "(cls, idx_cfg: Dict) ->
TrackNetConfig", "role": "Builds config from config.json `indexing.tracknet` sub-dict (keys:
wsl_distro, repo_dir, conda_sh, conda_env, weights_path, queue_length, stage_in_wsl,
wsl_stage_dir, timeout_sec)."}, {"name": "to_wsl_mnt_path", "kind": "func", "sig": "(win_path: str) ->
str", "role": "Windows path C:\\x -> /mnt/c/x; POSIX/~ paths returned unchanged. Shared module-
level helper imported by
wasb_runner."}, {"name": "TrajectoryPoint", "kind": "dataclass", "sig": "frame:int, visible:bool,
x:float, y:float", "role": "One per-frame shuttle sample; the cross-module trajectory element
type."}, {"name": "TrackNetRunner", "kind": "class", "sig": "(cfg: TrackNetConfig)", "role": "Runs
TrackNet predict.py in WSL and converts its CSV into action
windows."}, {"name": "TrackNetRunner._wsl_bash", "kind": "func", "sig": "(self, inner_cmd: str) ->
subprocess.CompletedProcess", "role": "Runs `wsl -d <distro> bash -c 'source conda.sh &&
<inner>`, capture_output, text, timeout=(timeout_sec or
None)."}, {"name": "TrackNetRunner.healthcheck", "kind": "func", "sig": "(self) ->
Tuple[bool, str]", "role": "Verifies WSL env activates + TF imports + GPU count; returns (ok,
msg)."}, {"name": "TrackNetRunner.run_predict", "kind": "func", "sig": "(self, video_win_path: str,
output_win_dir: str) -> Optional[str]", "role": "Stages video, runs predict.py, returns Windows
path to `<stem>_predict.csv` or
None."}, {"name": "TrackNetRunner._stage_video", "kind": "func", "sig": "(self, video_win_path: str,
base: str) -> Optional[str]", "role": "cp video into wsl_stage_dir via /mnt; returns WSL dest path
(echoes OK
sentinel)."}, {"name": "TrackNetRunner.parse_trajectory_csv", "kind": "func", "sig": "@staticmethod
(csv_win_path: str) -> List[TrajectoryPoint]", "role": "MODEL-AGNOSTIC: DictReader over
Frame,Visibility,X,Y; skips malformed rows; returns frame-sorted points."}, {"name": "TrackNetRunn
er.trajectory_to_action_windows", "kind": "func", "sig": "@staticmethod (points, fps, frame_width,
chunk_start=0.0, velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0, chunk_index=None)
-> List[Dict]", "role": "MODEL-AGNOSTIC WINDOWING BRAIN: marks visible+high-velocity frames
active, merges runs within merge_gap, drops short windows; emits candidate-rally
dicts."}], "contracts": ["Trajectory CSV header/column order (consumed): Frame,Visibility,X,Y.
Visibility==1 means visible; X,Y are pixel coords.", "TrajectoryPoint fields: frame:int,
visible:bool, x:float, y:float.", "Action-window dict (PRODUCED, downstream contract):
{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 always), chunk_index:Optional[int]}. start/end =
group_frame/fps + chunk_start.", "config.json indexing.tracknet keys: wsl_distro, repo_dir,
conda_sh, conda_env, weights_path, queue_length, stage_in_wsl, wsl_stage_dir,
timeout_sec.", "run_predict returns Windows path `<output_win_dir>/<stem>_predict.csv`
(predict.py's fixed naming) or None on any failure."], "edges": ["run_predict -> WSL
src/predict.py (--video_path --model_weights --output_dir --queue_length); reads CSV back via

```

```

/mnt, no copy.", "trajectory_to_action_windows -> (re-exported by)
wasb_runner.WasbRunner.trajectory_to_action_windows (shared windowing).", "parse_trajectory_csv
-> (re-exported by) wasb_runner.WasbRunner.parse_trajectory_csv.", "to_wsl_mnt_path -> imported
by wasb_runner for path translation + staging."], "gotchas": ["weights_path is REQUIRED and
defaults to '' -> run_predict logs an error and returns None if unset.", "predict.py only accepts
.mp4/.avi; other extensions are rejected before any WSL call.", "velocity is normalised:
dist/frame_width * fps/dframes; a visible==False frame RESETS prev (breaks the track) so motion
across an occlusion gap is never counted.", "Defensive defaults: fps<=0 -> 30.0, frame_width<=0
-> 1280.0 (silent fallbacks that skew velocity if real values are wrong).", "track_id_count is
hard-coded 1 (single shuttle) ? kept only for window-shape parity with the multi-track YOLO
segmenter.", "timeout_sec=0 means NO timeout (passed as None); a hung WSL call blocks
forever.", "Staging detects success only via an 'OK' echo sentinel in stdout, not just
returncode.", "Imports no TF ? all model code stays in WSL by design (keeps Windows env
clean)."]], {"path": "backend/detectors/wasb_runner.py", "role": "Windows-side adapter for the WASB-
SBDT backend: stages video + wasb_infer.py into WSL, runs inference in the `wasb` conda env, and
inherits the model-agnostic CSV/windowing from
TrackNetRunner."}, {"symbols": [{"name": "WasbConfig", "kind": "dataclass", "sig": "distro='Ubuntu',
repo_dir='~/models/WASB-SBDT', conda_sh, conda_env='wasb', weights_path='~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar', sport='badminton',
wsl_stage_dir='~/clips', timeout_sec=0", "role": "Resolved settings for a WASB WSL
run."}, {"name": "WasbConfig.from_indexing_cfg", "kind": "func", "sig": "(cls, idx_cfg: Dict) ->
WasbConfig", "role": "Builds config from config.json `indexing.wasb` (keys: wsl_distro, repo_dir,
conda_sh, conda_env, weights_path, sport, wsl_stage_dir,
timeout_sec)."}, {"name": "WasbRunner", "kind": "class", "sig": "(cfg:
WasbConfig)", "role": "Orchestrates the WSL WASB inference subprocess from
Windows."}, {"name": "WasbRunner._wsl_bash", "kind": "func", "sig": "(self, inner_cmd: str) ->
subprocess.CompletedProcess", "role": "Same `wsl -d bash -c 'source conda.sh && ...'` pattern as
TrackNetRunner (NOT shared ?
duplicated)."}, {"name": "WasbRunner.healthcheck", "kind": "func", "sig": "(self) ->
tuple", "role": "Verifies the `wasb` env imports torch and sees CUDA; returns (ok,
msg)."}, {"name": "WasbRunner._stage_video", "kind": "func", "sig": "(self, video_win_path, base) ->
Optional[str]", "role": "cp video into wsl_stage_dir (OK sentinel); returns WSL
dest."}, {"name": "WasbRunner._stage_infer_script", "kind": "func", "sig": "(self) ->
bool", "role": "Copies wasb_infer.py into <repo_dir>/src/ so it can import WASB's
detectors/trackers/data loaders."}, {"name": "WasbRunner.run_predict", "kind": "func", "sig": "(self,
video_win_path, output_win_dir) -> Optional[str]", "role": "Stages script+video, runs
wasb_infer.py --video, returns Windows path `<stem>_wasb.csv` or
None."}, {"name": "WasbRunner.parse_trajectory_csv", "kind": "func", "sig": "="
staticmethod(TrackNetRunner.parse_trajectory_csv)", "role": "Re-export of the shared CSV parser
(identical trajectory
shape)."}, {"name": "WasbRunner.trajectory_to_action_windows", "kind": "func", "sig": "="
staticmethod(TrackNetRunner.trajectory_to_action_windows)", "role": "Re-export of the shared
windowing brain."}, {"name": "_INFER_WIN", "kind": "const", "sig": "abspath of sibling
wasb_infer.py", "role": "Windows source path of the inference wrapper staged into
WSL."}], "contracts": ["Produces the same Frame, Visibility, X, Y CSV and same action-window dict
shape as TrackNetRunner (inherited statics).", "config.json indexing.wasb keys: wsl_distro,
repo_dir, conda_sh, conda_env, weights_path, sport, wsl_stage_dir, timeout_sec.", "run_predict
returns Windows path `<output_win_dir>/<stem>_wasb.csv` (wasb_infer.py is invoked with --out
pointing at the /mnt view of that path) or None.", "WASB candidate dict shape passed
model->tracker (defined/used inside WSL): {'xy': np.array([x,y]), 'score': float, 'scale':
int}.", "wasb_infer.py CLI contract this caller depends on: --video, --frames_out_dir, --weights,
--sport, --out.", "edges": ["imports to_wsl_mnt_path, TrackNetRunner from tracknet_runner.py
(path xlate + re-exported windowing).", "run_predict -> WSL src/wasb_infer.py (the staged
inference wrapper) with --video/--frames_out_dir/--weights/--sport/--out.", "_stage_infer_script
-> copies _INFER_WIN into <repo_dir>/src so wasb_infer can import WASB
modules."], "gotchas": ["Unlike TrackNet, run_predict does NOT pass the resumable-cache flags
(--cache-dir/--flush-every/--fresh/--batch-size/--num-workers); wasb_infer falls back to its own
defaults (cache next to --out, flush_every=1000, batch=8, workers=4).", "wasb_infer.py MUST be
re-staged each run (_stage_infer_script) because it lives in the WASB repo's src/ to satisfy
relative imports ? editing the Windows copy has no effect until re-staged.", "weights_path
default points to a badminton checkpoint; sport must match the weights/dataset config or WASB
Hydra config will mismatch.", "Healthcheck only checks torch+CUDA import, not that WASB modules
or Hydra configs are importable.", "_wsl_bash is duplicated here rather than reused from
TrackNetRunner ? drift risk if the WSL invocation convention changes.", "timeout_sec=0 -> no
timeout (hung GPU run blocks forever)."]], {"path": "backend/detectors/wasb_infer.py", "role": "Runs

```

```

INSIDE WSL `wasb` env: WASB shuttle inference over a frame folder with a reboot-resumable GPU
detector cache, emitting a Frame,Visibility,X,Y trajectory
CSV.",{"symbols":[{"name":"CACHE_VERSION","kind":"const","sig":"= 1","role":"Cache schema
version; mismatch invalidates the cache."},{name":"_DET_FILE /
_MANIFEST_FILE","kind":"const","sig":"'det_raw.jsonl' / 'manifest.json',"role":"Cache filenames
inside the cache dir."},{name":"_build_cfg","kind":"func","sig":"(weights: str, sport: str) ->
cfg","role":"Composes WASB eval config via Hydra (overrides dataset=<sport>, model=wasb,
detector.model_path, runner.device=cuda,
runner.gpus=[0])."}, {"name":"_frame_id","kind":"func","sig":"(path: str) -> int","role":"Best-
effort integer frame id from filename digits (frame_000180.png -> 180; -1 if
none)."}, {"name":"default_cache_dir","kind":"func","sig":"(out_csv: str) -> str","role":"Stable
cache dir `<out_stem>_wasbcache` NEXT TO the output (survives reboots; never
%TEMP%)."}, {"name":"_atomic_write_text","kind":"func","sig":"(path: str, text: str) ->
None","role":"Write to .tmp + flush + fsync + os.replace ? crash-safe
write."}, {"name":"_det_plain","kind":"func","sig":"(det) -> list","role":"Serialize a detector
candidate {'xy','score','scale'} -> [x,y,score,scale] JSON
list."}, {"name":"_dets_to_tracker","kind":"func","sig":"(plain_list) ->
list[dict]","role":"Inverse of _det_plain; rebuilds dicts with xy as np.array (tracker does
np.linalg.norm on it)."}, {"name":"write_manifest /
read_manifest","kind":"func","sig":"(cache_dir[, manifest]) -> None / dict[None]","role":"Atomic
write / tolerant read of manifest.json (returns None on
missing/corrupt)."}, {"name":"manifest_compatible","kind":"func","sig":"(manifest, *, frames_dir,
weights, sport, frames_in, frames_total) -> bool","role":"Cache reusable only if version +
frames_dir(abspath) + weights + sport + frames_in + frames_total all
match."}, {"name":"reset_cache","kind":"func","sig":"(cache_dir: str) -> None","role":"Delete
det_raw.jsonl + manifest.json for a clean
start."}, {"name":"load_and_clean_cache","kind":"func","sig":"(cache_dir: str) ->
(windows_done:int, det_by_fid: dict[int,list])","role":"Reads longest CONTIGUOUS valid prefix
(w==line index) of det_raw.jsonl, rebuilds per-frame candidate accumulation, REWRITES file to
that clean prefix so a crash-truncated trailing line can't corrupt future
appends."}, {"name":"_append_windows","kind":"func","sig":"(fh, objs) -> None","role":"Append
window JSONL records then flush+fsync (durable, bounded
loss)."}, {"name":"_atomic_write_trajectory","kind":"func","sig":"(out_csv, rows) ->
None","role":"Crash-safe write of Frame,Visibility,X,Y CSV (header + rows) via
.tmp+fsync+replace."}, {"name":"run","kind":"func","sig":"(frames_dir, weights, out_csv,
sport='badminton', limit=0, batch_size=8, num_workers=4, log_every_batches=50, cache_dir=None,
flush_every=1000, fresh=False) -> str","role":"Core: build sliding windows, resume/run the GPU
detector pass with checkpointing, then re-run the CPU tracker over the full ordered cache ->
trajectory CSV."}, {"name":"extract_frames","kind":"func","sig":"(video_path, frames_dir) ->
str","role":"cv2 decode video to <i:08d>.png; idempotent (skips if existing count >= container
frame count)."}, {"name":"main","kind":"func","sig":"()","role":"argparse CLI:
--frames_dir/--video, --weights, --out, --sport, --limit, --batch-size, --num-workers, --log-
every-batches, --cache-dir, --flush-every, --fresh."}], "contracts":[{"det_raw.jsonl line shape
(one window/line): {\\"w\\": <window_index>, \\"f\\": [[frame_id, [[x,y,score,scale], ...]], ...}].
Lines MUST be contiguous: line N has w==N."}, {"manifest.json shape: {version, frames_dir(abspath),
weights, sport, frames_in, frames_total, windows_total, windows_done}.", "Detector candidate dict
(in-WSL, model->tracker): {'xy': np.array([x,y],float), 'score': float, 'scale': int}; plain
form [x,y,score,scale].", "det_by_fid: dict frame_id(int) -> list of [x,y,score,scale]
accumulated across every window containing that frame, in window order.", "Output CSV (PRODUCED):
header Frame,Visibility,X,Y; row = (fid, 1 if tracker visi else 0, x or -1, y or -1).", "Sliding
window = frames_in consecutive frames; windows_total = len(frames) - frames_in + 1; sample =
{'frames':[paths], 'annos':[{frame_path,
center=Center(is_visible=False,x=-1,y=-1)}}].", "Tracker.update returns {'visi','x','y'} per
frame; det_by_fid iterated in sorted frame-id order."}, {"edges":["run -> WASB src modules:
detectors.build_detector, trackers.build_tracker,
dataloaders.build_img_transforms/build_seq_transforms, dataloaders.dataset_loader.ImageDataset,
utils.Center (all imported lazily inside run; only resolvable from WASB-SBDT/src).", "run ->
detector.run_tensor(imgs, trans) (per-batch raw window detections) and
tracker.refresh()/tracker.update(...) (stateful per-frame trajectory).", "run -> default_cache_di
r/read_manifest/manifest_compatible/load_and_clean_cache/reset_cache/_append_windows/write_manif
est/_atomic_write_trajectory (the resumable-cache machinery).", "_build_cfg ->
hydra.compose(config_name='eval') with overrides (requires WASB configs/ on the working
dir).", "Invoked by wasb_runner.WasbRunner.run_predict via the staged copy; produces the CSV that
tracknet_runner.parse_trajectory_csv reads."], "gotchas":["Resume split is deliberate: GPU
detector pass is checkpointed (resumable, ~1 forward/window, ~21k for a 6-min clip); the CPU

```

tracker is STATEFUL/order-dependent and CANNOT resume mid-stream ? it is ALWAYS fully re-run over the ordered cache (cheap, deterministic). Losing the CSV costs seconds, not the GPU hour.", "Cache contiguity rule: load_and_clean_cache stops at the first line where obj['w'] != line index OR a JSONDecodeError (crash-truncated tail), then rewrites the file to only the good prefix ? so a half-written final line is healed on next start.", "Cache auto-invalidates if frames_dir(abspath)/weights/sport/frames_in/frames_total or CACHE_VERSION changes (manifest_compatible) -> falls back to a fresh run; --fresh forces it.", "dets_to_tracker MUST return xy as a numpy array (tracker does vector math / np.linalg.norm); plain JSON lists would crash the tracker.", "Hard requirement: len(frames) >= frames_in or SystemExit; frames are sorted glob of *.png + *.jpg (frame ordering relies on zero-padded names ? _frame_id parses digits).", "runner.gpus=[0] is hard-coded for a single-GPU box (default WASB config assumes multiple).", "extract_frames uses cv2 PNG encoding which is SLOW for big clips; docstring recommends host-side ffmpeg + --frames_dir instead.", "flush_every/log_every_batches are clamped to >=1 (max(1,...)); manifest windows_done is updated on each flush so resume granularity == flush boundary.", "Pure-Python cache helpers (no torch/numpy at module top) are intentionally unit-testable; torch/WASB imports happen lazily inside run() so importing the module on Windows won't fail."], [{"path": "backend/detectors/__init__.py", "role": "Package marker / docstring only ? declares that detectors shell out to WSL-quarantined TF/Torch envs to keep the main Windows env free of TensorFlow/CUDA."}, {"symbols": [], "contracts": ["No exports; pure docstring. References docs/DECISIONS.md ADR-001 for the WSL2 quarantine decision."], "edges": [], "gotchas": ["Empty of code ? does NOT re-export the runners, so callers must import backend.detectors.tracknet_runner / wasb_runner directly."]}], [{"subsystem": "Eval & calibration", "summary": "Segmenter-agnostic evaluation/calibration stack for rally detection. The core is `metrics.py`: temporal-IoU greedy one-to-one matching of predicted (start,end) intervals vs ground truth, yielding precision/recall/F1/mean-IoU/boundary-errors. Everything else layers on it: `harness.py` scores DB-stored predictions (candidates vs final stages); `batch.py` aggregates harness reports corpus-wide (micro/macro); `calibration.py` is the pure overfit-guard layer (k-fold CV, leave-one-video-out, improvement_report) parameterised by a `predict\_fn(config)->intervals`; `calibrate\_wasb.py` + `export\_wasb\_segments.py` are CLI drivers wiring WASB trajectory CSVs through `TrackNetRunner` into that calibration layer; `annotations.py` parses hand-labelled GT CSVs; `regression.py` compares DB predictions to a snapshot JSON baseline via annotation providers; `training.py` + `classifier.py` build/train the offline learned segmenter (numpy logistic regression) that aims to replace paid Gemini refinement. Two interval conventions coexist: a generic `start,end` GT CSV (annotations.py) and a richer golden/slicer schema with `start\_time/end\_time` (calibrate_wasb/export). All metric/calibration/training/classifier code is pure (no I/O, no GPU); side-effects live in the CLI drivers and separate run_*.py orchestrators."}, {"files": [{"path": "backend/eval/metrics.py", "role": "Pure temporal interval-matching metrics ? the scoring primitive every other file builds on. No I/O, deterministic, unit-tested."}, {"symbols": [{"name": "Interval", "kind": "const", "sig": "Tuple[float, float]", "role": "Type alias: (start, end) seconds, start < end. The universal interval shape across the subsystem."}, {"name": "interval_iou", "kind": "func", "sig": "(a: Interval, b: Interval) -> float", "role": "Temporal IoU = overlap/union; 0.0 if no overlap or either degenerate."}, {"name": "Match", "kind": "dataclass", "sig": "pred_index:int, gt_index:int, iou:float", "role": "One matched predicted<->GT pair."}, {"name": "MatchResult", "kind": "dataclass", "sig": "matches:List[Match], unmatched_pred_indices:List[int], unmatched_gt_indices:List[int]", "role": "Matching outcome; unmatched_pred=FPs, unmatched_gt=FNs."}, {"name": "match_intervals", "kind": "func", "sig": "(preds, gts, iou_threshold=0.5) -> MatchResult", "role": "Greedy one-to-one matching by descending IoU; ties broken by indices for determinism."}, {"name": "Score", "kind": "dataclass", "sig": "iou_threshold, num_pred, num_gt, true_positives, false_positives, false_negatives, precision, recall, f1, mean_iou, mean_start_error, mean_end_error; .as_dict()->dict", "role": "Aggregate metrics for one preds-vs-GT evaluation. .as_dict() is the JSON/cross-module wire shape."}, {"name": "score", "kind": "func", "sig": "(preds, gts, iou_threshold=0.5, match_result=None) -> Score", "role": "Compute P/R/F1 + mean matched IoU + boundary errors; pass match_result to avoid re-matching."}, {"name": "pr_curve", "kind": "func", "sig": "(preds, gts, thresholds=None) -> List[Score]", "role": "Sweep IoU thresholds (default 0.1..0.9 step 0.1)."}], "contracts": ["Interval = (start_seconds, end_seconds) float tuple, start < end.", "Score.as_dict() returns all 12 Score fields as a flat dict ? consumed by harness.report_to_dict, batch.aggregate (keys true_positives/false_positives/false_negatives/mean_iou/f1), calibrate_wasb, regression."], "MatchResult.matches indices are into the preds/gts lists passed to match_intervals (not stable across re-sorts).", "mean_iou/mean_start_error/mean_end_error are 0.0 when there are no matches (not NaN)."}, {"edges": [], "gotchas": ["Greedy (not optimal/Hungarian) matching: maximises per-pair IoU first, so global TP count can in pathological cases be sub-optimal ? acceptable here, kept for determinism."], "score()'s mean_start_error/mean_end_error index

```

preds[m.pred_index]/gts[m.gt_index] ? the caller MUST pass the SAME preds/gts lists used to
build match_result, or boundary errors are silently wrong.", "interval_iou returns 0.0 (not
error) for degenerate intervals; an interval with end<=start is silently scored as a non-
match."}], {"path": "backend/eval/annotations.py", "role": "Load/validate hand-annotated ground-
truth rally intervals from a simple `start,end` CSV (one file per
video).", "symbols": [{"name": "parse_timestamp", "kind": "func", "sig": "(value: str) ->
float", "role": "Parse decimal seconds OR colon MM:SS / HH:MM:SS to float seconds; raises
ValueError on garbage."}, {"name": "load_annotations", "kind": "func", "sig": "(csv_path: str) ->
List[Interval]", "role": "Read GT CSV -> intervals sorted by start; raises ValueError on malformed
rows / start>=end (fail loud)."}, {"name": "write_scaffold", "kind": "func", "sig": "(csv_path: str)
-> None", "role": "Write an empty `start,end`-header file for hand-filling."}], "contracts": ["GT
CSV schema: 2 columns `start,end`; optional header row (`start`/`start_time` first cell auto-
skipped); blank lines and `#` comments ignored; timestamps decimal-seconds or colon
MM:SS/HH:MM:SS, mixable.", "Returns List[Interval] sorted ascending by start; raises (never
silently drops) on start>=end or unparseable timestamp.", "Distinct from the golden/slicer schema
used by calibrate_wasb/export_wasb_segments (which use start_time/end_time
columns)."], "edges": ["load_annotations -> metrics.Interval (produces the Interval shape consumed
downstream by harness/regression)"], "gotchas": ["Colon parse folds segments most-significant-
first (total = total*60 + part), so it accepts both MM:SS and HH:MM:SS but rejects >3
segments.", "This is the GENERIC GT loader; the WASB golden pipeline uses a different loader
(calibrate_wasb.load_golden_gts) with start_time/end_time columns ? don't conflate
them."}], {"path": "backend/eval/calibration.py", "role": "Pure measurement layer for calibration:
quantify precision lift AND guard against overfitting (k-fold CV, leave-one-video-out).
Segmenter-agnostic via
predict_fn.", "symbols": [{"name": "Config", "kind": "const", "sig": "Any", "role": "Opaque config object
(e.g. a thresholds tuple/dict) ? this module never inspects
it."}, {"name": "PredictFn", "kind": "const", "sig": "Callable[[Config], List[Interval]]", "role": "The
injected segmenter: maps a config to predicted intervals. The seam that makes calibration
segmenter-agnostic."}, {"name": "pct_improvement", "kind": "func", "sig": "(before: float, after:
float) -> float", "role": "Percent change; returns inf if before<=0 and after>0, else
0.0."}, {"name": "evaluate", "kind": "func", "sig": "(preds, gts, iou_threshold=0.5) ->
Score", "role": "Thin pass-through to
metrics.score."}, {"name": "kfold_indices", "kind": "func", "sig": "(n: int, k: int) ->
List[Tuple[List[int], List[int]]]", "role": "Deterministic round-robin k-fold (train_idx, test_idx)
splits; k capped to >=2 and <=n."}, {"name": "select_best", "kind": "func", "sig": "(predict_fn,
configs, gts, metric='f1', iou_threshold=0.5) -> Tuple[Config, float]", "role": "Pick config
maximising a Score attribute against
gts."}, {"name": "cross_validate", "kind": "func", "sig": "(predict_fn, configs, gts, k=5,
metric='recall', iou_threshold=0.5) -> Dict", "role": "Within-video k-fold overfit guard; reports
mean_test/mean_train/generalization_gap/picked_configs."}, {"name": "leave_one_video_out", "kind": "
func", "sig": "(videos, configs, metric='f1', iou_threshold=0.5) -> Dict", "role": "LOVO CV across
videos ? the honest cross-video generalization
number."}, {"name": "improvement_report", "kind": "func", "sig": "(predict_fn, baseline_config,
tuned_config, gts, iou_threshold=0.5) -> Dict", "role": "The '+X% precision' headline: baseline vs
tuned on same GT."}], "contracts": ["cross_validate returns {metric, folds, mean_test, std_test,
mean_train, generalization_gap, picked_configs} OR {error, n} if <2 folds
possible.", "leave_one_video_out videos arg: List[{name: str, 'predict_fn': PredictFn,
'gts': List[Interval]}]; returns {metric, mean_held_out, std_held_out, generalization_gap,
per_video: [{video, held_out, train_pooled, config}]} OR {error, n} if <2
videos.", "improvement_report returns {baseline: {precision, recall, f1, mean_iou}, tuned: {...},
precision_delta_pts, precision_delta_pct, f1_delta_pct}.", "select_best returns (config,
metric_value); metric must be a valid Score attribute name (getattr)."], "edges": ["evaluate/selec
t_best/cross_validate/leave_one_video_out/improvement_report -> metrics.score (all scoring
delegates to the metrics primitive)"], "gotchas": ["cross_validate defaults to metric='recall'
DELIBERATELY: predictions are whole-video windows, so scoring them against a fold SUBSET of GT
would count other rallies' windows as false positives ? precision/F1 are not well-defined per-
fold. Use leave_one_video_out for precision/F1 generalization.", "generalization_gap >~0.1 (train
>> test/held_out) is the overfit alarm.", "select_best/LOVO tie-break: select_best keeps first
config on strict > (earliest in list wins ties); LOVO uses max(...) which also keeps first
max.", "kfold_indices silently drops folds where train or test is empty; with very few rallies
you can get fewer than k folds (or {error} from
cross_validate)."], {"path": "backend/eval/calibrate_wasb.py", "role": "CLI driver: tune WASB
rally-windowing thresholds against golden rallies and print '+X%' lift + honest cross-val
recall. Wires TrackNetRunner ->
calibration.", "symbols": [{"name": "Config", "kind": "const", "sig": "Tuple[float, float, float] =

```

```
(velocity_thresh, merge_gap, min_window_duration)", "role": "The WASB threshold tuple swept during
calibration."}, {"name": "BASELINE", "kind": "const", "sig": "Config = (0.02, 2.0, 2.0)", "role": "The
WasbConfig / trajectory_to_action_windows defaults ? the 'before' operating
point."}, {"name": "load_golden_gts", "kind": "func", "sig": "(csv_path: str) ->
List[Interval]", "role": "Parse golden CSV (start_time/end_time cols, case/space-insensitive) ->
sorted intervals, silently skipping bad
rows."}, {"name": "make_predict_fn", "kind": "func", "sig": "(trajectory_csv, fps, frame_width) ->
Callable[[Config], List[Interval]]", "role": "Parse trajectory ONCE, return
predict_fn(cfg)->windows. The PredictFn for
calibration."}, {"name": "default_grid", "kind": "func", "sig": "()" ->
List[Config]", "role": "5x3x3=45-config sweep: vt in {0.005, .01, .02, .03, .05}, mg in {1,2,3}, mwd in
{1,2,3}."}, {"name": "run", "kind": "func", "sig": "(trajectory_csv, labels_csv, fps, frame_width,
iou=0.5) -> dict", "role": "Full calibration: baseline eval, grid select_best(f1),
improvement_report (optimistic), cross_validate (honest); prints + returns {baseline, best_cfg,
fit, cv}."}, {"name": "main", "kind": "func", "sig": "()" -> None", "role": "argparse CLI: --trajectory
--labels --fps(59.94) --frame-width(1920) --iou(0.5)."}], "contracts": ["Golden CSV schema
(consumed): columns start_time, end_time (float seconds); fieldnames lowercased+stripped; rows
with non-float/empty/e<=s are silently skipped.", "Trajectory CSV schema (consumed via
TrackNetRunner): Frame, Visibility, X, Y.", "trajectory_to_action_windows returns dicts with at
least 'start' and 'end' keys (mapped to intervals).", "run() return dict: {baseline:
Score.as_dict(), best_cfg: Config, fit: improvement_report dict, cv: cross_validate
dict}."], "edges": ["make_predict_fn ->
detectors/tracknet_runner.TrackNetRunner.parse_trajectory_csv (parse trajectory once) and
.trajectory_to_action_windows (cfg->windows)", "run -> calibration.select_best /
improvement_report / cross_validate / evaluate (the actual calibration
math)"], "gotchas": ["HONESTY footgun (documented): the grid is selected on the FULL GT, so
improvement_report on that same GT is an OPTIMISTIC upper bound; trust the cross-validated held-
out RECALL instead. leave_one_video_out needs a 2nd labelled video.", "load_golden_gts swallows
malformed rows silently (continue) ? unlike annotations.load_annotations which raises. Different
failure philosophy for the two GT loaders.", "Defaults fps=59.94 / frame-width=1920 are baked
into the CLI; wrong values silently mis-scale velocity thresholds.", "BASELINE must stay in sync
with the real WasbConfig/trajectory_to_action_windows defaults ? it's a hardcoded
duplicate."}], {"path": "backend/eval/export_wasb_segments.py", "role": "CLI driver: turn a cached
WASB trajectory + tuned config into actual rally segments (golden/slicer CSV schema) plus a per-
rally tuned-vs-golden comparison, for manual eyeballing / clip
cutting."}, {"symbols": [{"name": "TUNED", "kind": "const", "sig": "Config = (0.05, 1.0,
1.0)", "role": "Tuned operating point from the first per-video calibration (F1 0.588->0.746). CLI-
overridable."}, {"name": "SEG_FIELDS", "kind": "const", "sig": "[rally_number, start_time, end_time,
duration, start_mmss, end_mmss]", "role": "Golden/slicer CSV column order for exported
segments."}, {"name": "COMP_FIELDS", "kind": "const", "sig": "[type, ref, start_time, end_time, mmss,
status, iou, pred_start, pred_end, start_err_s, end_err_s]", "role": "Comparison CSV column
order."}, {"name": "seconds_to_mmss", "kind": "func", "sig": "(t: float) -> str", "role": "Format
seconds -> 'M:SS.mmm' scrub
timestamp."}, {"name": "segments_to_rows", "kind": "func", "sig": "(segments: List[Interval]) ->
List[Dict[str, str]]", "role": "Number segments 1..N in time order; emit SEG_FIELDS
rows."}, {"name": "write_segments_csv", "kind": "func", "sig": "(path, segments) -> str", "role": "Write
segments in golden schema (feeds
rally_slicer.load_rally_labels)."}, {"name": "build_comparison", "kind": "func", "sig": "(preds, gts,
iou_threshold=0.5) -> dict", "role": "Golden-centric per-rally comparison rows + Score
(matched/MISS per golden,
pred_extra=FP)."}, {"name": "write_comparison_csv", "kind": "func", "sig": "(path, comparison) ->
str", "role": "Write COMP_FIELDS rows."}, {"name": "run", "kind": "func", "sig": "(trajectory_csv, fps,
frame_width, cfg=TUNED, out_csv='output/wasb_tuned_segments.csv', labels_csv=None, iou=0.5,
baseline=False, comparison_out=None) -> dict", "role": "Export tuned (and optional baseline)
segments; if labels given, write+print tuned-vs-golden
comparison."}, {"name": "main", "kind": "func", "sig": "()" -> None", "role": "argparse CLI incl.
--velocity-thresh/--merge-gap/--min-duration, --baseline, --video/--slice for clip
cutting."}], "contracts": ["Exported segment CSV = SEG_FIELDS, consumed by
tools.rally_slicer.load_rally_labels (the schema doubles as slicer input).", "Comparison CSV =
COMP_FIELDS; status in {matched, MISS, false_positive}; type in {golden, pred_extra}; ref is
1-based.", "build_comparison returns {'score': Score, 'rows': List[Dict[str, str]]}.", "run()
return dict: {tuned_cfg, num_tuned, out_csv} (+ num_baseline/baseline_csv if --baseline; +
comparison(Score.as_dict())/comparison_csv if labels)."], "edges": ["run/make_predict_fn ->
calibrate_wasb.make_predict_fn, load_golden_gts, BASELINE (reuse trajectory->segments + golden
loader)", "build_comparison -> metrics.match_intervals / score (same matching as the
```

```

evaluator)","_maybe_slice -> tools.rally_slicer.slice_rallies (cut one padded clip per tuned
rally; lazy import)"],["gotchas":["TUNED is a hardcoded operating point from ONE video's first
calibration ? not auto-loaded from calibrate_wasb output; re-tuning won't update it unless you
pass CLI flags.","--slice requires --video (raises SystemExit otherwise); slicing is a lazy
import of rally_slicer (heavier deps).","Output CSVs default under output/; _ensure_parent
makedirs the parent. Comparison file name defaults to <out>_vs_golden.csv, baseline to
<out>_baseline.csv via _with_suffix."]],{"path":"backend/eval/batch.py","role":"Pure corpus-
level aggregation across a collector export manifest (Phase 3). Path resolution, stage
selection, micro/macro metric aggregation ? no I/O/GPU (that's in run_phase3_eval.py).","symbols
":[{"name":"_FINAL_ONLY_SEGMENTERS","kind":"const","sig":"'motion','gemini'"},"role":"Segmenter
s with no raw-candidate stage; scoring 'candidates' for them is
meaningless."],{"name":"resolve_video_path","kind":"func","sig":"(local_path: Optional[str],
videos_root: str) -> Optional[str]"},"role":"Normalise collector manifest paths (Windows seps,
leading ./) and resolve relative to videos_root; None if
empty."],{"name":"default_stages_for","kind":"func","sig":"(segmenter: str, requested='auto') ->
List[str]"},"role":"Pick stages to score: auto -> both for two-stage detectors, final-only for
_FINAL_ONLY_SEGMENTERS; explicit request
overrides."],{"name":"aggregate","kind":"func","sig":"(per_video: List[dict], stages: List[str])
-> Dict[str, dict]"},"role":"Pool per-video stage scores into micro P/R/F1 (pool TP/FP/FN), TP-
weighted mean IoU, macro mean F1."],{"name":"format_aggregate","kind":"func","sig":"(agg:
Dict[str, dict]) -> str"},"role":"Human-readable corpus summary table."}],{"contracts":["aggregate
input per_video: list of harness.report_to_dict dicts ? each must have
['stages'][stage]['score'] with keys
true_positives/false_positives/false_negatives/mean_iou/f1.","aggregate output per stage:
{videos, tp, fp, fn, micro_precision, micro_recall, micro_f1, macro_f1,
mean_iou}.","default_stages_for requested values:
'auto'|'candidates'|'final'|'both'."],"edges":["aggregate <- consumes the dict shape produced by
harness.report_to_dict (cross-module contract, not a direct import)"],["gotchas":["micro mean_iou
is TP-weighted (sum(mean_iou*tp)/sum(tp)) ? videos with 0 TP contribute 0, not NaN.","Videos
missing a stage in their report are skipped per-stage (n_videos counts only present), so
'videos' can differ between stages.","Pure module by design ? the actual manifest walking / DB
writes / GPU run live in run_phase3_eval.py (not in this subsystem's file
list)."]],{"path":"backend/eval/harness.py","role":"Score the pipeline's DB-stored detected
rallies vs ground truth. Reads predictions from SQLite (no pipeline re-run, no API cost) for two
stages: candidates (raw) vs final (AI-confirmed).","symbols":[{"name":"STAGES","kind":"const","s
ig":"'candidates','final'"},"role":"The two scoreable prediction
stages."],{"name":"StageReport","kind":"dataclass","sig":"stage:str,
pred_intervals:List[Interval], score:Score, match_result:MatchResult,
pr_curve:List[Score]"},"role":"Per-stage scoring
result."],{"name":"EvalReport","kind":"dataclass","sig":"video_id:str, iou_threshold:float,
gt_intervals:List[Interval], stages:Dict[str,StageReport]"},"role":"Per-video evaluation result
across stages."],{"name":"get_predictions","kind":"func","sig":"(db: Database, video_id: str,
stage: str, detector=None) -> List[Interval]"},"role":"Fetch one stage's predicted intervals from
the DB; raises ValueError on unknown stage."],{"name":"evaluate","kind":"func","sig":"(db,
video_id, gt_intervals, stages, iou_threshold=0.5, detector=None, with_pr_curve=False) ->
EvalReport"},"role":"Score requested stages for one video vs
GT."],{"name":"report_to_dict","kind":"func","sig":"(report: EvalReport) -> dict"},"role":"JSON-
serialisable view {video_id, iou_threshold, num_ground_truth, stages:{stage:{score, pr_curve}}}
? the shape batch.aggregate
consumes."],{"name":"format_report_text","kind":"func","sig":"(report, show_failures=False) ->
str"},"role":"Console table; show_failures lists missed (FN) and spurious (FP) interval
timestamps."}],{"contracts":["get_predictions: stage 'candidates' ->
db.query_candidate_segments(video_id, detector=detector); stage 'final' ->
db.query_play_segments(video_id, {}); both rows need 'start_time'/'end_time' (cast float, sorted
by start).","report_to_dict output is the exact input contract for batch.aggregate
(stages[stage]['score'] = Score.as_dict()).","video_id is the file MD5 key used in the DB (per
regression.py docs)."],"edges":["get_predictions -> database.Database.query_candidate_segments /
query_play_segments (read predictions from SQLite)","evaluate -> metrics.match_intervals / score
/ pr_curve (scoring)","report_to_dict -> Score.as_dict (serialisation); consumed by
batch.aggregate","get_predictions <- imported by regression.regress and
regression.regress_with_provider (reuses the same prediction
fetch)"],["gotchas":["query_play_segments is called with an empty filter dict {} ? fetches ALL
final segments for the video; no per-detector filtering on the final stage (detector arg only
affects candidates).","pr_curve is only computed when with_pr_curve=True (default off) ?
otherwise StageReport.pr_curve is []."]],{"path":"backend/eval/training.py","role":"Pure data

```

layer (Phase 4 / ADR-004) for the offline learned segmenter: label candidate windows by GT match and extract YOLO feature vectors -> (X, y, intervals). No model/IO

here.", "symbols": [{"name": "YOLO_FEATURE_NAMES", "kind": "const", "sig": "[duration, peak_velocity, mean_velocity, active_frame_count, track_id_count]", "role": "Feature ordering for YOLO candidate rows; persisted into the model for inspectability."}, {"name": "extract_yolo_features", "kind": "func", "sig": "(row: Dict) -> List[float]", "role": "Build the 5-d YOLO feature vector from a candidate row's deserialized `stats` (missing fields -> 0.0)."}, {"name": "label_candidates", "kind": "func", "sig": "(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]", "role": "1 if window matched one-to-one to a GT rally (same matching as evaluator), else 0."}, {"name": "build_training_set", "kind": "func", "sig": "(candidate_rows, gt_intervals, iou_threshold=0.5, feature_fn=extract_yolo_features) -> Tuple[List[List[float]], List[int], List[Interval]", "role": "Glue: candidate rows + GT -> (X, y, intervals); feature_fn pluggable for a WASB substrate."}], "contracts": ["Candidate row dict (from db.query_candidate_segments): must have 'start_time', 'end_time'; 'stats' is a JSON-deserialized dict (DB layer does the deserialization) with optional peak_velocity/mean_velocity/active_frame_count/track_id_count.", "build_training_set returns (X: List[List[float]] feature matrix, y: List[int] 0/1 labels, intervals: List[Interval]); X and intervals are row-aligned with candidate_rows; y aligned with intervals.", "feature_fn signature: Callable[[Dict], List[float]] ? must emit vectors in a fixed, consistent order (paired with feature_names)."], "edges": ["label_candidates -> metrics.match_intervals (uses the SAME matching the evaluator scores TP with, so labels == scoring)", "build_training_set -> classifier.LogisticRegression.fit (downstream consumer of (X, y); feature_names = YOLO_FEATURE_NAMES)", "gotchas": ["Labels are deliberately defined by the evaluator's IoU matching so the trained classifier optimises exactly what's scored ? keep iou_threshold consistent between training and eval.", "extract_yolo_features assumes row['stats'] already a dict (None -> {}); if a caller passes a raw JSON string this silently produces all-zero features.", "No model/training/IO in this file by design ? the trainer/segmenter is separate and chosen after Phase 3 candidate-recall results."], {"path": "backend/eval/classifier.py", "role": "Dependency-free numpy logistic regression (the learned segmenter baseline). Standardized features, balanced-class GD, sigmoid, JSON (de)serialization ? no sklearn/pickle."}, {"name": "LogisticRegression", "kind": "class", "sig": "(lr=0.1, epochs=1000, l2=1e-3)", "role": "Self-contained logistic-regression model with fit/predict/predict_proba/save/load."}, {"name": "LogisticRegression.fit", "kind": "func", "sig": "(X, y, feature_names=None, class_weight='balanced') -> self", "role": "Standardize, balanced-weight GD; raises ValueError on empty/non-2D X."}, {"name": "LogisticRegression.predict_proba", "kind": "func", "sig": "(X) -> np.ndarray", "role": "Sigmoid probabilities; raises RuntimeError if unfitted."}, {"name": "LogisticRegression.predict", "kind": "func", "sig": "(X, threshold=0.5) -> np.ndarray[int]", "role": "Thresholded 0/1 predictions."}, {"name": "LogisticRegression.to_dict / from_dict", "kind": "func", "sig": "to_dict()->dict ; from_dict(d)->LogisticRegression", "role": "JSON-safe (de)serialization of weights+standardization+hyperparams."}, {"name": "LogisticRegression.save / load", "kind": "func", "sig": "save(path)->None ; load(path)->LogisticRegression (classmethod)", "role": "Persist/restore as small human-readable JSON."}], "contracts": ["Model JSON schema: {model:'logreg', lr, epochs, l2, w:[...]|None, b:float, mean:[...]|None, std:[...]|None, feature_names:[...]|None}."], "fit() stores mean/std for standardization (std==0 -> 1.0 guard); predict_proba/predict re-standardize incoming X with the SAME mean/std ? feature ORDER at predict time must match fit time.", "Expects X = 2D array (n_samples x n_features) e.g. training.build_training_set's X; y = 0/1."], "edges": ["LogisticRegression.fit <- consumes (X, y, feature_names) produced by training.build_training_set / YOLO_FEATURE_NAMES"], "gotchas": ["class_weight='balanced' default ? rally windows are heavily imbalanced (many spurious candidates per real rally); pass class_weight!='balanced' to disable.", "feature_names are metadata only (not enforced at predict) ? silent corruption if you feed features in a different order than training; mean/std re-standardization will mis-scale.", "Sigmoid input clipped to [-30,30] for overflow safety; no convergence check ? fixed epochs (1000) GD, tune lr/epochs/l2 manually.", "numpy-only by intent: sklearn is NOT installed in this project; keep the fit/predict/save/load interface if swapping models."], {"path": "backend/eval/regression.py", "role": "Regression runner: score DB-stored predictions vs golden GT (via an annotation provider) and flag drops beyond tolerance against a snapshotted JSON baseline. The CI/golden-set guard."}, {"name": "DEFAULT_BASELINE_PATH", "kind": "const", "sig": "output/regression_baseline.json", "role": "Default per-video baseline snapshot file."}, {"name": "DEFAULT_TOLERANCE", "kind": "const", "sig": "0.05", "role": "Allowed P/R

```

drop before flagging a
regression."}, {"name": "RegressionResult", "kind": "dataclass", "sig": "video_id, stage,
iou_threshold, precision, recall, f1, baseline_precision:Optional, baseline_recall:Optional,
tolerance, regressed:bool, reasons:List[str]; .as_dict()", "role": "Outcome of one regression
check."}, {"name": "load_baselines", "kind": "func", "sig": "(path=DEFAULT_BASELINE_PATH) -> Dict[str,
dict]", "role": "Load baseline snapshot JSON ({ if
absent)."}, {"name": "save_baseline", "kind": "func", "sig": "(video_id, stage, precision, recall, f1,
path=...) -> None", "role": "(Re)snapshot current numbers as 'known good' after a legit
improvement."}, {"name": "regress", "kind": "func", "sig": "(db, video_id, ground_truth,
stage='final', iou_threshold=0.5, detector=None, tolerance=0.05, baseline_path=...) ->
RegressionResult", "role": "Score stored preds vs GT, compare to baseline; regressed if precision
OR recall drops > tolerance."}, {"name": "regress_with_provider", "kind": "func", "sig": "(db,
video_id, video_ref, tier='file', stage='final', ..., guideline=None, sport='badminton',
provider_kwargs=None) -> RegressionResult", "role": "End-to-end: get GT from annotation provider
tier, then regress."}, {"name": "format_result", "kind": "func", "sig": "(r: RegressionResult) ->
str", "role": "One-line [REGRESSED|ok] verdict + reasons."}], "contracts": [{"Baseline JSON:
{f'video_id': {stage}: {video_id, stage, precision, recall, f1}} (key via _baseline_key,
sorted-keys on write).", "ground_truth = List[metrics.Interval]; video_id = file MD5 keying DB
predictions; video_ref = whatever the provider expects (CSV path for FileProvider, id/URL for
vendor).", "tier is an annotation_registry key ('file' today; 'vendor'/'auto' later);
provider.annotate(video_ref, guideline=, sport=) -> List[Interval].", "If no baseline exists,
regressed=False with an explanatory reason (caller may then save_baseline).", "edges": ["regress
-> harness.get_predictions (fetch stored predictions) and metrics.score (score vs
GT)", "regress_with_provider -> annotations.annotation_registry.get(tier,
**provider_kwargs).annotate(...) (obtain GT from a provider; backend.annotations, NOT
backend.eval.annotations)"], "gotchas": ["Imports annotation_registry from backend.annotations
(the provider package), NOT backend/eval/annotations.py ? two different 'annotations' modules;
easy to confuse.", "Regression fires on precision OR recall drop > tolerance; F1 is reported but
NOT used as a gate.", "Only F1 reported, but baseline stores precision/recall/f1 ? the gate
ignores stored f1.", "First run on a video silently passes (no baseline) ? you must explicitly
save_baseline to start guarding it; defaults write under output/."]}], {"subsystem": "Stitcher,
tools, utils, validators", "summary": "Post/pre-processing utilities for the rally pipeline. The
stitcher (VideoCompiler) trims selected (start,end) segments out of a raw video with padding and
concat them into a highlight reel via two ffmpeg passes (re-encode per clip, then stream-copy
concat). rally_slicer is a standalone CLI that slices individual labeled rallies out of an
annotated match using the golden CSV schema, sharing the compiler's padding constants; its
window math (compute_clip_windows) is pure/unit-tested. utils/video.py and utils/geometry.py are
thin shared helpers (ffprobe duration, ffmpeg chunk extraction; bbox
center/distance/velocity/IoU). validators/ is a pluggable VideoValidator framework: an abstract
base + ValidationResult (pydantic) + a global registry that runs all validators in sequence
threading a shared `context` dict; concrete validators gate ingested clips on format/resolution-
fps/blur/scene-cut/pts-continuity/relevance. All ffmpeg/ffprobe calls shell out to external
binaries (must be on PATH); OpenCV-based validators read frames directly. Config flows in via a
`config` dict (config.json), validator tunables under
config["validation"].", "files": [{"path": "backend/stitcher/compiler.py", "role": "Trims selected
(start,end) segments from a raw video with begin/end padding and concatenates them into one
highlight reel via
ffmpeg.", "symbols": [{"name": "VideoCompiler", "kind": "class", "sig": "__init__(output_dir: str,
end_padding=1.0, begin_padding=0.5)", "role": "Stateful compiler; makedirs(output_dir) on init;
holds padding defaults."}, {"name": "VideoCompiler._parse_time", "kind": "func", "sig": "@staticmethod
(val: int|float|str) -> float", "role": "Normalizes timestamps: numeric, plain string, or colon
'MM:SS'/'HH:MM:SS'; warns + returns 0.0 on unparseable. Mirrors gemini.py
parse_time."}, {"name": "VideoCompiler._get_video_duration", "kind": "func", "sig": "(video_path: str)
-> float", "role": "ffprobe format=duration; returns 0.0 on any
failure."}, {"name": "VideoCompiler.compile_highlights", "kind": "func", "sig": "(raw_video_path: str,
segments: List[Tuple[float,float]], output_filename: str) -> str", "role": "Main entry:
validate/clamp each segment, trim to temp clips (re-encode), concat-copy to output_path; returns
output path."}], "contracts": ["Input segments: List[Tuple[start, end]] where start/end may be
int|float|str (parsed via _parse_time).", "skipped_segments tuple layout: (idx:int, start:float,
end:float, reason:str).", "Per-clip ffmpeg trim args: -ss/-to padded_start/padded_end, -c:v
libx264 -preset superfast -crf 22 -c:a aac -b:a 128k.", "Concat list file 'clips.txt' lines: file
'<path>' (single-quotes escaped as '\\').", "Concat ffmpeg: -f concat -safe 0 -c copy (assumes
all clips share identical encode settings).", "Returns: absolute-ish output_path str
(os.path.join(output_dir, output_filename)).", "edges": ["compile_highlights -> ffmpeg/ffprobe
(external binaries via subprocess; must be on PATH)", "padding defaults (0.5/1.0) mirrored by

```

```

backend/tools/rally_slicer.py BEGIN_PADDING/END_PADDING (kept in sync by convention, not
import)"},"gotchas":["Per-clip pass RE-ENCODES (libx264/superfast/crf22); concat pass uses -c
copy and relies on every clip having byte-identical codec params ? a clip clamped differently
still concats fine because all use the same encode args."],"padded_start uses begin_padding
(default 0.5) NOT zero; start is clamped to >=0 and start>=end segments are skipped, not
errored."],"If video_duration probe fails (0.0) NO boundary validation happens ? out-of-range
segments pass through to ffmpeg."],"TemporaryDirectory(dir=self.output_dir) creates temp clips
inside output_dir (not system tmp); cleaned on context exit."],"Raises ValueError (empty
segments), FileNotFoundError (missing source), RuntimeError (zero valid clips OR concat
failure). Caller must handle."],"Path escaping only handles single quotes; exotic paths in the
concat list could still break."],"Logs to stdout via print() with [COMPILER ...] prefixes, not
logging module."}],{"path":"backend/tools/rally_slicer.py","role":"Standalone CLI: slice
individual labeled rally clips out of an annotated match video using the golden labels CSV, with
compiler-matched padding."},"symbols":[{"name":"BEGIN_PADDING","kind":"const","sig":"=
0.5","role":"Lead-in padding; matches
VideoCompiler.begin_padding."},"{"name":"END_PADDING","kind":"const","sig":"= 1.0","role":"Lead-
out padding; matches
VideoCompiler.end_padding."},"{"name":"ClipWindow","kind":"dataclass","sig":"rally: str, start:
float, end: float","role":"One padded+clamped output window; rally kept as str to allow inserted
ids like '7.5'."},"{"name":"load_rally_labels","kind":"func","sig":"(csv_path: str) -> Dict[str,
Tuple[float,float]]","role":"Parse golden CSV -> rally_number(str) -> (start,end) secs; skips
rows missing/degenerate times."},"{"name":"compute_clip_windows","kind":"func","sig":"(labels,
rally_numbers: List[str], begin_pad=0.5, end_pad=1.0, video_duration=None) ->
List[ClipWindow]","role":"Pure (no ffmpeg): apply padding + clamp; supports ['all']; skips
unknown ids; unit-tested."},"{"name":"slice_rallies","kind":"func","sig":"(video_path,
labels_csv, rally_numbers, out_dir, begin_pad=0.5, end_pad=1.0) -> List[str]","role":"Probe
duration, compute windows, ffmpeg-trim each rally; returns written clip
paths."},"{"name":"main","kind":"func","sig":"() -> None","role":"argparse CLI: --video --labels
--rallies --out-dir --begin-pad --end-pad."}],{"contracts":["Labels CSV schema (golden): columns
rally_number, start_time, end_time, optional start_mmss, end_mmss. Header names
lowercase+stripped; _read_time prefers decimal-seconds col, falls back to mm:ss."],"Rally
numbers are strings throughout (preserves inserted ids like '7.5')."","rally_numbers == ['all']
(case-insensitive) -> every labeled rally in start-time order."],"Output filename:
<video_stem>__rally_<n>.mp4 where '.' in rally id is replaced with '_' (e.g. rally 7.5 ->
__rally_7_5.mp4)."","Per-clip ffmpeg: -ss/-to w.start/w.end -c:v libx264 -preset superfast -crf
22 -c:a aac -b:a 128k (same recipe as compiler)."","ClipWindow.start/end are rounded to 3
decimals."],"edges":["_read_time -> backend/eval/annotations.parse_timestamp (parse decimal or
mm:ss timestamp strings)","_probe_duration / slice_rallies -> ffmpegprobe/ffmpeg (external
binaries)"],"gotchas":["load_rally_labels silently skips rows where start/end missing or
end<=start (e.g. not-yet-filled MISSED golden rows) ? no warning."],"compute_clip_windows
silently drops unknown rally ids; caller (slice_rallies raises only if ZERO windows) gets no
per-id warning."],"Padding constants duplicate VideoCompiler's via copy not import ? keep in sync
manually."},"compute_clip_windows is the pure/testable core; slice_rallies is the only ffmpeg-
touching function."],"Module-level imports backend.eval.annotations ? running as a script needs
the package importable (run via -m or with repo root on
PYTHONPATH)."}],{"path":"backend/eval/video.py","role":"Shared ffmpeg/ffmpegprobe helpers: probe
duration and extract a single chunk (copy or re-
encode)."},"symbols":[{"name":"get_video_duration","kind":"func","sig":"(video_path: str) ->
float","role":"ffmpegprobe format=duration; 0.0 on failure (logs
error)."},"{"name":"extract_video_chunk","kind":"func","sig":"(video_path, start_time: float,
duration: float, output_path, reencode=False) -> bool","role":"ffmpeg -ss/-t extract;
reencode=True -> libx264/ultrafast/crf23/aac (frame-accurate), else -c copy (keyframe-
aligned)."}],{"contracts":["extract_video_chunk uses -t DURATION (length), not -to END ? differs
from compiler/slicer which use -ss/-to."],"Returns bool success (True/False), not the
path."],"edges":["get_video_duration -> ffmpegprobe; extract_video_chunk -> ffmpeg (external
binaries)"],"gotchas":["Default reencode=False uses stream-copy which cuts at the NEAREST
KEYFRAME (drift) ? pass reencode=True for short/frame-accurate clips."],"duration arg is a length
in seconds (-t), easy to confuse with an end timestamp."],"Uses logging module (logger =
logging.getLogger(__name__)), unlike compiler's
print()."}],{"path":"backend/eval/geometry.py","role":"Pure bbox geometry helpers (center,
distance, velocity, IoU) used by detection/tracking
logic."},"symbols":[{"name":"get_center","kind":"func","sig":"(bbox: (x1,y1,x2,y2)) ->
(cx,cy)","role":"Box center."},"{"name":"get_distance","kind":"func","sig":"(p1, p2) ->
float","role":"Euclidean distance between two (x,y)
points."},"{"name":"get_velocity","kind":"func","sig":"(bbox1, bbox2, fps: float) ->

```

```

float", "role": "Center displacement * fps = pixels/sec between consecutive
frames."}, {"name": "get_velocity_normalised", "kind": "func", "sig": "(bbox1, bbox2, fps,
frame_width) -> float", "role": "Velocity as fraction of frame width/sec (resolution-independent
threshold)."}, {"name": "calculate_iou", "kind": "func", "sig": "(boxA, boxB) ->
float", "role": "Intersection-over-Union of two boxes; 0.0 if no overlap."}], "contracts": ["All
bboxes are (x1, y1, x2, y2) tuples in pixel coords.", "fps is the EFFECTIVE fps after frame-
skipping (caller's responsibility).", "get_velocity_normalised falls back to RAW pixel velocity
when frame_width <= 0."], "edges": [], "gotchas": ["calculate_iou uses the +1 pixel convention (xB-
xA+1) ? areas are inclusive-pixel, slightly inflates IoU vs the continuous (no +1) convention;
be consistent if comparing thresholds across modules.", "No clamping/validation of bbox ordering
(assumes x2>=x1, y2>=y1)."]}, {"path": "backend/validators/base.py", "role": "Validator framework:
abstract VideoValidator, pydantic ValidationResult, and the shared global ValidationRegistry
that runs all validators with a shared
context.", "symbols": [{"name": "ValidationResult", "kind": "dataclass", "sig": "pydantic BaseModel:
validator_name: str, passed: bool, message: str, details: Dict[str, Any] = {}", "role": "Uniform
result returned by every validator."}, {"name": "VideoValidator", "kind": "class", "sig": "ABC:
@property name, @property description, validate(video_path, config, context) ->
ValidationResult", "role": "Contract all concrete validators
implement."}, {"name": "ValidationRegistry", "kind": "class", "sig": "register(validator);
run_all(video_path, config) -> List[ValidationResult]", "role": "Ordered list of validators; runs
each in sequence threading a fresh shared context
dict."}, {"name": "registry", "kind": "registry", "sig": "= ValidationRegistry()", "role": "Module-level
singleton other modules register validators into."}], "contracts": ["validate signature is fixed:
(video_path: str, config: Dict, context: Dict) -> ValidationResult.", "context dict is created
empty PER run_all() call and shared across validators in execution order ? used to pass
ffprobe_meta (set by format_check) downstream (resolution_fps_check reads it).", "config is the
parsed config.json dict; validators read config['validation'] (and config['sport']).", "On
exception run_all wraps it as ValidationResult(passed=False, message='Validator threw...') ?
note: this construct omits details, so it defaults to {}."], "edges": ["registry consumed by all
backend/validators/*.py concrete validators (they are registered into it elsewhere)", "run_all
context dict is the cross-validator channel: format.py writes context['ffprobe_meta'],
resolution_fps.py reads it"], "gotchas": ["Validators run in REGISTRATION ORDER and share context
? resolution_fps_check FAILS unless format_check ran earlier in the same run_all (ordering
dependency).", "run_all swallows all exceptions into a failing result, so a buggy validator never
aborts the batch but always reports passed=False.", "registry is a global singleton ? import
side-effects determine which validators are active; double-import could double-
register."]}, {"path": "backend/validators/blur.py", "role": "Validator: samples 15 evenly-spaced
frames and rejects footage whose mean Laplacian variance (focus metric) is below
threshold.", "symbols": [{"name": "BlurValidator", "kind": "class", "sig": "VideoValidator;
name='blur_check'", "role": "Laplacian-variance sharpness gate via OpenCV."}], "contracts": ["config
key: config['validation']['min_blur_threshold'] (default 100.0).", "details:
{avg_laplacian_variance, min_laplacian_variance, max_laplacian_variance,
sampled_frames_count}.", "Samples num_samples=15 frames; step = max(1,
total_frames//15).", "edges": ["validate -> cv2.VideoCapture/Laplacian (OpenCV; reads frames
directly, no ffmpeg)", "gotchas": ["cap.set(POS_FRAMES) seeking is unreliable on some
codecs/containers ? sampled frame may not be exactly frame_idx.", "Higher Laplacian variance =
sharper; the check is avg_variance < threshold => fail (a LOW score is bad).", "Returns
passed=False (not skip) if the video can't be opened or no frames read ? treats unreadable as a
quality failure."]}, {"path": "backend/validators/format.py", "role": "Validator: ffprobe-probes the
container, requires MP4, and caches full ffprobe JSON into context for downstream
validators.", "symbols": [{"name": "FormatValidator", "kind": "class", "sig": "VideoValidator;
name='format_check'", "role": "MP4 container + integrity check; populates
context['ffprobe_meta']."}], "contracts": ["Writes context['ffprobe_meta'] = full ffprobe JSON
(keys: 'streams' list, 'format' dict) ? consumed by resolution_fps.py.", "ffprobe args:
-select_streams v:0 -show_format -show_streams -of json.", "details: {format_name,
duration:float, size_bytes:int, codec_name}.", "Pass condition: 'mp4' substring in
format.format_name (lowercased).", "edges": ["validate -> ffprobe (external); writes
context['ffprobe_meta'] consumed by
backend/validators/resolution_fps.ResolutionFPSValidator"], "gotchas": ["MUST run before
resolution_fps_check (it is the producer of ffprobe_meta) ? registration order
matters.", "Container check is a substring match on format_name; ffprobe reports MP4 as
'mov,mp4,m4a,3gp,3g2,mj2' which contains 'mp4' so it passes (also matches some non-mp4
names).", "On CalledProcessError stuffs raw e.stderr into
details['error']."]}, {"path": "backend/validators/pts_continuity.py", "role": "Validator: extracts
keyframe PTS via ffprobe and rejects videos with backward jumps or large timeline gaps

```

```
(editing/splicing
signal).", "symbols": [{"name": "PTSContinuityValidator", "kind": "class", "sig": "VideoValidator;
name='pts_continuity_check'", "role": "Keyframe PTS gap/backward-jump
analysis."}], "contracts": ["ffprobe args: -select_streams v:0 -show_entries packet=pts_time,flags
-of csv=p=0; parses lines 'pts_time,flags', keeps only flags containing 'K'
(keyframe).", "details: {keyframes_count, max_keyframe_gap_seconds, backward_jumps_detected,
first_keyframe, last_keyframe}.", "Fail if backward_jumps>0 OR max_gap>10.0s. <2 keyframes =>
pass ('too short').", "Thresholds are HARDCODED (8.0s mentioned in comment, 10.0s used in code) ?
NOT read from config."], "edges": ["validate -> ffprobe packet inspection
(external)"], "gotchas": ["10.0s gap threshold and backward-jump rule are hardcoded constants,
ignoring config ? unlike other validators.", "Only keyframes (flag 'K') are inspected; non-
keyframe PTS reordering (B-frames) is invisible here.", "Comment says 8.0s but the actual code
threshold is 10.0s ? comment/code
drift."], {"path": "backend/validators/relevance.py", "role": "Validator: samples 10 frames and
rejects videos lacking enough court-colored ('green') pixels per the sport profile's HSV
bounds.", "symbols": [{"name": "RelevanceValidator", "kind": "class", "sig": "VideoValidator;
name='relevance_check'", "role": "HSV court-color ratio gate; sport-profile
driven."}], "contracts": ["Reads sport via config['sport'] (default 'badminton'); fetches
sport_profile.get_relevance_config().", "HSV bounds resolution order:
config['validation']['relevance'][k] -> sport_rel_cfg[k] -> hardcoded default. Keys:
court_green_lower (def [35,40,40]), court_green_upper (def [85,255,255]), court_pixels_min_ratio
(def 0.05).", "details: {avg_green_court_ratio, max_green_court_ratio, min_green_court_ratio,
green_threshold}.", "Frames downsampled to 320x240 before HSV masking; ratio =
green_pixels/total_pixels.", "Fail if avg_green_ratio < min_court_ratio."], "edges": ["validate ->
backend.sports.sport_registry.get(sport_name).get_relevance_config() (HSV bounds per
sport)", "validate -> cv2 (OpenCV frame sampling/HSV/inRange)"], "gotchas": ["Imports
backend.sports lazily inside validate(); KeyError from unknown sport => sport_rel_cfg={} (falls
through to user-config/hardcoded defaults, does NOT fail).", "Default HSV range targets GREEN
courts ? badminton courts are often green/blue; tune via sport profile or
config['validation']['relevance'] for blue/wood courts.", "Local h,w from frame.shape are unused
(frame is hard-resized to 320x240
regardless)."]}, {"path": "backend/validators/resolution_fps.py", "role": "Validator: reads cached
ffprobe metadata and enforces minimum resolution and frame
rate.", "symbols": [{"name": "ResolutionFPSValidator", "kind": "class", "sig": "VideoValidator;
name='resolution_fps_check'", "role": "Min-resolution + min-fps gate from context
ffprobe_meta."}], "contracts": ["Reads context['ffprobe_meta']['streams'][0] for
width/height/r_frame_rate (NOT re-probing).", "r_frame_rate parsed as 'num/den' fraction (e.g.
'30000/1001').", "config keys: config['validation']['min_resolution_width'] (def 1280),
min_resolution_height (def 720), min_fps (def 29.9).", "details: {width, height,
fps}.", "edges": ["validate -> consumes context['ffprobe_meta'] produced by
backend/validators/format.FormatValidator"], "gotchas": ["HARD DEPENDENCY: fails with 'Run
format_check validator first' if context['ffprobe_meta'] absent ? registration/order
coupling.", "Only inspects streams[0]; if format_check selected v:0 this is the video stream, but
a different probe could put audio first.", "min_fps default 29.9 (not 30) to tolerate 29.97
NTSC."], {"path": "backend/validators/scene_cut.py", "role": "Validator: samples ~2 fps, compares
consecutive HSV histograms, and rejects videos exceeding a cuts-per-minute limit
(edited/compiled
signal).", "symbols": [{"name": "SceneCutDensityValidator", "kind": "class", "sig": "VideoValidator;
name='scene_cut_check'", "role": "Histogram-correlation scene-cut density
gate."}], "contracts": ["Samples at ~2 fps: sample_interval = max(1, int(fps/2)); caps at first
100 sampled frames.", "HSV hist over channels [0,1], bins [50,60], ranges [0,180,0,256],
NORM_MINMAX normalized; HISTCMP_CORREL.", "cut_threshold = 0.6 (HARDCODED): correlation < 0.6
between consecutive samples => one cut.", "config key:
config['validation']['max_scene_cuts_per_minute'] (default 0.5).", "details: {cuts_detected,
sampled_frames, cuts_per_minute}. Fail if cuts_per_minute > max.", "edges": ["validate ->
cv2.VideoCapture/calcHist/compareHist (OpenCV; reads frames directly)"], "gotchas": ["100-frame
cap means only the first ~50s (at 2fps) is analyzed ? cuts later in a long video are missed;
cuts/min is extrapolated from that window.", "cut_threshold 0.6 is hardcoded (not config-driven);
only max_scene_cuts_per_minute is tunable.", "Repeated cap.set(POS_FRAMES) seeking per sample is
slow/unreliable on long-GOP codecs.", "Default max 0.5 cuts/min is strict ? assumes a continuous
raw recording, will reject any lightly-edited clip."]]}, {"subsystem": "Sports, providers,
annotations, DB", "summary": "Three parallel pluggable-registry subsystems plus the SQLite
persistence layer. (1) SPORTS: a SportProfile ABC + BadmintonSportProfile supply the per-sport
Gemini system prompt (with timestamp-format rules and JSON output schema) and visual-relevance
config; a module-level sport_registry singleton auto-registers badminton. (2) PROVIDERS: an
```

AIVideoProvider ABC + GeminiProvider implement video->(segments, metrics) analysis via Google GenAI (upload/poll/generate/delete lifecycle); provider_registry singleton auto-registers 'gemini'. (3) ANNOTATIONS: an AnnotationProvider ABC (submit/poll/fetch/annotate) yields ground-truth rally (start,end) intervals ? the exact shape backend.eval.metrics consumes ? with FileProvider (CSV via backend.eval.annotations.load_annotations) and AutomatedProvider (model-agnostic oracle stub with an injectable labeler + lineage guard); annotation_registry singleton, providers register as import side-effects. (4) DATABASE: a SQLite Database class with videos / play_segments / events / candidate_segments tables, JSON-blob extensibility columns, FK CASCADE, and a PRAGMA user_version migration ladder. All three registries follow the same pattern: dict keyed by a provider/profile `name`, with a get() that raises KeyError listing available keys. The (start,end) interval shape is the cross-cutting contract tying annotations to the eval harness.", "files": [{"path": "backend/sports/base.py", "role": "ABC defining the per-sport profile interface (prompt + relevance config).", "symbols": [{"name": "SportProfile", "kind": "class", "sig": "ABC", "role": "Abstract base for sport-specific analysis config."}, {"name": "SportProfile.name", "kind": "func", "sig": "@property -> str", "role": "Sport identifier, e.g. 'badminton'; used as registry key."}, {"name": "SportProfile.get_prompt", "kind": "func", "sig": "(chunk_duration: float = 0.0) -> str", "role": "Full AI system prompt incl. output JSON schema."}, {"name": "SportProfile.get_relevance_config", "kind": "func", "sig": "()" -> dict, "role": "Visual validation params (court colors, thresholds)."}], "contracts": [{"name": "property is the registry key.", "get_relevance_config() returns a dict; concrete shape defined by badminton.py (court_green_lower/upper, court_pixels_min_ratio).", "get_prompt(chunk_duration) must produce a prompt that elicits the JSON array schema defined in badminton.py."}], "edges": [], "gotchas": ["Pure interface; all three methods abstract ? concrete shapes live in badminton.py, not here."]}, {"path": "backend/sports/badminton.py", "role": "Badminton SportProfile: the actual Gemini system prompt + court-color relevance config.", "symbols": [{"name": "BadmintonSportProfile", "kind": "class", "sig": "extends SportProfile", "role": "Concrete badminton profile."}, {"name": "BadmintonSportProfile.name", "kind": "func", "sig": "@property -> str = 'badminton'", "role": "Registry key."}, {"name": "BadmintonSportProfile.get_prompt", "kind": "func", "sig": "(chunk_duration: float = 0.0) -> str", "role": "Returns long CV-extraction prompt; prepends a duration-bounds preamble when chunk_duration>0."}, {"name": "BadmintonSportProfile.get_relevance_config", "kind": "func", "sig": "()" -> dict, "role": "Returns HSV court-green bounds + min pixel ratio."}], "contracts": [{"name": "Prompt instructs model to emit a JSON ARRAY of objects with keys: start_time (float, TOTAL DECIMAL SECONDS), end_time (float), shuttle_exchange_count (int), shot_count_confidence ('Low'|'Medium'|'High'). NO markdown.", "get_relevance_config() dict: {'court_green_lower': [35,40,40], 'court_green_upper': [85,255,255], 'court_pixels_min_ratio': 0.05} ? HSV triplets as lists.", "chunk_duration>0 prepends 'IMPORTANT VIDEO CONTEXT...' clamping timestamps to [0, chunk_duration].", "edges": [{"name": "BadmintonSportProfile -> backend/sports/base.py.SportProfile (subclass)", "role": "Prompt repeatedly warns against MM:SS / concatenated-minute timestamps ? downstream pipeline assumes total decimal seconds. Mis-parsed timestamps are a known failure mode the prompt actively defends against.", "shot_count_confidence is a STRING enum ('Low'/'Medium'/'High'), not numeric.", "The model is asked NOT to wrap output in markdown; gemini.py sets response_mime_type='application/json' to enforce this."}], "path": "backend/sports/__init__.py", "role": "SportRegistry class + module-level sport_registry singleton (auto-registers badminton).", "symbols": [{"name": "SportRegistry", "kind": "class", "sig": "", "role": "Maps sport name -> SportProfile subclass."}, {"name": "SportRegistry.register", "kind": "func", "sig": "(profile_cls: Type[SportProfile])", "role": "Instantiates cls to read .name, then stores the CLASS keyed by name."}, {"name": "SportRegistry.get", "kind": "func", "sig": "(name: str) -> SportProfile", "role": "Returns a NEW instance; raises KeyError listing available sports if missing."}, {"name": "SportRegistry.list_available", "kind": "func", "sig": "()" -> Dict[str, str], "role": "name -> docstring (or fallback) map."}, {"name": "sport_registry", "kind": "registry", "sig": "SportRegistry()", "role": "Module singleton; constructor auto-registers BadmintonSportProfile."}], "contracts": [{"name": "_registry: Dict[str, Type[SportProfile]] (stores classes, not instances).", "get() returns a fresh instance each call."}], "edges": [{"name": "SportRegistry.__init__ -> backend/sports/badminton.py.BadmintonSportProfile (auto-register)", "role": "register -> backend/sports/base.py.SportProfile (type bound)", "gotchas": ["register() instantiates the profile twice over its lifecycle (once at register to read .name, once per get()) ? profiles must be cheap/side-effect-free to construct.", "list_available uses class __doc__; BadmintonSportProfile has no docstring so it falls back to '<Name> sport profile'."]}], {"path": "backend/providers/base.py", "role": "ABC for AI video providers ? the

```

video->(segments, metrics) analysis
interface.", "symbols": [{"name": "AIVideoProvider", "kind": "class", "sig": "ABC; __init__(api_key:
str, model_name: str)", "role": "Abstract multimodal video
analyzer."}, {"name": "AIVideoProvider.analyze_video", "kind": "func", "sig": "(video_path: str,
prompt: str, chunk_duration: float = 0.0, label: str = '') -> Tuple[list, dict]", "role": "Analyze
a video file with a prompt; return (segments, metrics)."}, {"contracts": ["Returns Tuple[list,
dict]: segments = list of dicts (e.g. {'start_time': ..., 'end_time': ...}); metrics dict =
{'upload_time', 'process_time', 'gen_time'} per gemini.py.", "Constructor stores self.api_key,
self.model_name."], "edges": [], "gotchas": ["chunk_duration and label are diagnostic/context-only
here; segment shape is provider/sport-defined, not enforced by the
ABC."]}, {"path": "backend/providers/gemini.py", "role": "GeminiProvider: uploads video to Google
GenAI, polls processing, generates JSON segments, deletes the
file."}, {"symbols": [{"name": "GeminiProvider", "kind": "class", "sig": "extends
AIVideoProvider", "role": "Google Gemini multimodal video
analyzer."}, {"name": "GeminiProvider.client", "kind": "func", "sig": "@property ->
genai.Client", "role": "Lazily constructs genai.Client(api_key) on first
use."}, {"name": "GeminiProvider.analyze_video", "kind": "func", "sig": "(video_path, prompt,
chunk_duration=0.0, label='') -> Tuple[list, dict]", "role": "4-step: upload -> poll PROCESSING ->
generate_content (JSON, temp 0) -> parse; always best-effort deletes the uploaded
file."}], "contracts": ["metrics dict keys: 'upload_time', 'process_time', 'gen_time' (floats,
seconds).", "Uses google.genai types.GenerateContentConfig(response_mime_type='application/json',
temperature=0.0).", "Returns ([, metrics) on ANY failure (upload, FAILED state, status-check
error, empty text, non-list JSON, parse error).", "Parsed result MUST be a JSON list (top-level
array) or it's rejected."], "edges": ["GeminiProvider -> backend/providers/base.py.AIVideoProvider
(subclass)", "client -> google.genai.Client (external SDK)", "analyze_video ->
google.genai.types.GenerateContentConfig (external)"], "gotchas": ["Polls Google processing with a
BLOCKING time.sleep(10) loop ? synchronous, can hang the calling thread for the full server-side
processing duration.", "On every exit path it tries to delete the uploaded file (best-effort,
swallows exceptions); upload failures return before any delete.", "Client is lazy:
auth/connection errors surface on first .client access, not at __init__ ? instantiation never
validates the api_key.", "temperature=0.0 for determinism; relies on response_mime_type to avoid
markdown-fenced output.", "Returns empty list (not an exception) on failure ? callers must
distinguish 'no rallies' from 'analysis failed' some other
way."]}, {"path": "backend/providers/__init__.py", "role": "ProviderRegistry class + module-level
provider_registry singleton (auto-registers
'gemini')."}, {"symbols": [{"name": "ProviderRegistry", "kind": "class", "sig": "", "role": "Maps provider
name -> AIVideoProvider
subclass."}, {"name": "ProviderRegistry.register", "kind": "func", "sig": "(name: str, provider_cls:
Type[AIVideoProvider])", "role": "Store class under an explicit
name."}, {"name": "ProviderRegistry.get", "kind": "func", "sig": "(name: str, api_key: str,
model_name: str) -> AIVideoProvider", "role": "Instantiate named provider with creds; raises
KeyError listing available if
missing."}, {"name": "ProviderRegistry.list_available", "kind": "func", "sig": "()" ->
Dict[str, str]", "role": "name -> docstring
map."}, {"name": "provider_registry", "kind": "registry", "sig": "ProviderRegistry()", "role": "Module
singleton; auto-registers GeminiProvider as 'gemini'."}], "contracts": ["_registry: Dict[str,
Type[AIVideoProvider]].", "get() injects api_key + model_name into the provider
constructor."], "edges": ["ProviderRegistry.__init__ -> backend/providers/gemini.py.GeminiProvider
(auto-register as 'gemini')"], "gotchas": ["Unlike SportRegistry (key derived from instance.name),
provider keys are passed EXPLICITLY to register() ? the string 'gemini' lives only here, not on
the class."]}, {"path": "backend/annotations/base.py", "role": "AnnotationProvider ABC
(submit/poll/fetch/annotate) + registry + status constants + Interval type for ground-truth
rally labels."}, {"symbols": [{"name": "Interval", "kind": "const", "sig": "Tuple[float,
float]", "role": "(start, end) seconds; identical to backend.eval.metrics.Interval ? the cross-
module contract."}, {"name": "QUEUED/RUNNING/DONE/FAILED", "kind": "const", "sig": "str =
'queued'|'running'|'done'|'failed'", "role": "Job lifecycle
statuses."}, {"name": "STATUSES", "kind": "const", "sig": "(QUEUED, RUNNING, DONE,
FAILED)", "role": "Valid poll() return
values."}, {"name": "AnnotationProvider", "kind": "class", "sig": "ABC", "role": "Abstract ground-truth
label source."}, {"name": "AnnotationProvider.name", "kind": "func", "sig": "@property ->
str", "role": "Unique registry key ('file', 'auto',
etc.)."}, {"name": "AnnotationProvider.submit", "kind": "func", "sig": "(video_ref: str, guideline:
Optional[str]=None, sport: str='badminton') -> str", "role": "Start a job; return opaque
job_id."}, {"name": "AnnotationProvider.poll", "kind": "func", "sig": "(job_id: str) ->
str", "role": "Return one of

```

```

STATUSES."}, {"name": "AnnotationProvider.fetch", "kind": "func", "sig": "(job_id: str) ->
List[Interval]", "role": "Return sorted (start,end) labels; should RAISE (not return []) if not
DONE/unavailable."}, {"name": "AnnotationProvider.annotate", "kind": "func", "sig": "(video_ref,
guideline=None, sport='badminton') -> List[Interval]", "role": "Sync convenience =
submit+poll+fetch; raises if job not immediately
DONE."}, {"name": "AnnotationProviderRegistry", "kind": "class", "sig": "", "role": "Maps provider name
-> AnnotationProvider
subclass."}, {"name": "AnnotationProviderRegistry.register", "kind": "func", "sig": "(provider_cls) ->
Type (decorator-friendly)", "role": "Key by provider().name; falls back to __name__ if
construction needs args."}, {"name": "AnnotationProviderRegistry.get", "kind": "func", "sig": "(name:
str, **kwargs) -> AnnotationProvider", "role": "Instantiate, passing kwargs to constructor (e.g.
labeler, lineage, csv_dir)."}, {"name": "annotation_registry", "kind": "registry", "sig": "AnnotationP
roviderRegistry()", "role": "Module singleton; populated by import side-effects of
file_provider/automated_provider."}], "contracts": ["Interval = Tuple[float, float] (start,end
seconds) ? the canonical shape shared with backend.eval.annotations.load_annotations and
backend.eval.metrics.", "poll() returns a STATUSES string.", "fetch() returns List[Interval]
sorted by start; contract says RAISE rather than return [] on unavailable.", "_providers:
Dict[str, Type[AnnotationProvider]]."], "edges": ["Interval mirrors
backend/eval/annotations.py.Interval and backend/eval/metrics.Interval (intentional shape parity
for the regression flow)."], "gotchas": ["register() instantiates the class with NO args to read
.name; providers requiring constructor args (none here do at construction) would fall back to
keying by __name__ instead of their intended name ? silent mis-keying risk.", "annotate() is for
SYNCHRONOUS providers only; on an async provider it raises once the job isn't immediately
DONE.", "Empty-vs-raise discipline matters: returning [] would silently zero out recall in eval
scoring."], {"path": "backend/annotations/file_provider.py", "role": "FileProvider: synchronous
ground-truth source reading pre-made start,end CSV files (default golden-set source / CI stand-
in).", "symbols": [{"name": "FileProvider", "kind": "class", "sig": "extends AnnotationProvider;
__init__(csv_dir: Optional[str]=None, ext: str='.csv')", "role": "CSV-backed label provider;
job_id IS the resolved CSV path."}, {"name": "FileProvider.name", "kind": "func", "sig": "@property ->
'file'", "role": "Registry key."}, {"name": "FileProvider.submit", "kind": "func", "sig": "(video_ref,
guideline=None, sport='badminton') -> str", "role": "Returns resolved CSV path; guideline/sport
ignored."}, {"name": "FileProvider.poll", "kind": "func", "sig": "(job_id) -> str", "role": "DONE if
file exists else FAILED."}, {"name": "FileProvider.fetch", "kind": "func", "sig": "(job_id) ->
List[Interval]", "role": "Loads via load_annotations; raises FileNotFoundError if
missing."}], "contracts": ["job_id == resolved CSV path.", "_resolve: existing path -> used as-is;
else <csv_dir>/<video_ref><ext> if csv_dir set; else video_ref unchanged.", "CSV format (per
backend.eval.annotations): 'start,end' rows, optional header, decimal-seconds OR MM:SS/HH:MM:SS,
'#'/blank lines skipped; rows validated (start<end) or it raises
ValueError."], "edges": ["FileProvider.fetch -> backend/eval/annotations.py.load_annotations
(parse + validate CSV into sorted intervals)", "module bottom ->
backend/annotations/base.py.annotation_registry.register(FileProvider) (import side-
effect)"], "gotchas": ["submit() does NO work ? it presumes the label CSV already exists on disk;
poll() only checks existence, never validates content.", "guideline and sport args are accepted
but ignored (a file has no notion of them).", "Registered as side-effect of import; must be
imported (via backend.annotations.__init__) for 'file' to appear in the
registry."], {"path": "backend/annotations/automated_provider.py", "role": "AutomatedProvider:
model-agnostic fast-tier 'oracle' annotation interface (GS-4 stub) with injectable labeler +
lineage guard. No real model
wired.", "symbols": [{"name": "Labeler", "kind": "const", "sig": "Callable[[str, Optional[str], str],
List[Interval]]", "role": "Type alias: (video_ref, guideline, sport) ->
intervals."}, {"name": "_not_configured", "kind": "func", "sig": "(video_ref, guideline, sport) ->
List[Interval]", "role": "Default labeler that RAISES NotImplementedError ? keeps the provider
constructible but fails loudly if used
unconfigured."}, {"name": "warn_if_same_lineage", "kind": "func", "sig": "(oracle_lineage:
Optional[str], production_lineage: Optional[str]) -> bool", "role": "Logs a warning + returns True
if oracle and production share a model family (encodes the 'oracle
rule')."}, {"name": "AutomatedProvider", "kind": "class", "sig": "extends AnnotationProvider;
__init__(labeler: Optional[Labeler]=None, lineage: str='unspecified')", "role": "Delegates
labeling to injected callable; in-process synchronous job
store."}, {"name": "AutomatedProvider.name", "kind": "func", "sig": "@property ->
'auto'", "role": "Registry
key."}, {"name": "AutomatedProvider.submit", "kind": "func", "sig": "(video_ref, guideline=None,
sport='badminton') -> str", "role": "Runs labeler NOW, sorts to (float,float) intervals, stashes
under job_id=video_ref."}, {"name": "AutomatedProvider.poll", "kind": "func", "sig": "(job_id) ->
str", "role": "DONE if job in store else

```

```

FAILED."},{ "name": "AutomatedProvider.fetch", "kind": "func", "sig": "(job_id) ->
List[Interval]", "role": "Returns stashed intervals; raises KeyError if unknown
job."}, {"contracts": ["Labeler signature: (video_ref:str, guideline:Optional[str], sport:str) ->
List[Interval].", "_jobs: Dict[str, List[Interval]] in-process store; job_id ==
video_ref.", "submit() normalizes intervals via sorted((float(s), float(e)) for s, e in
intervals).", "lineage attribute records oracle model family for warn_if_same_lineage
guarding."], "edges": ["module bottom ->
backend/annotations/base.py.annotation_registry.register(AutomatedProvider) (import side-
effect)"], "gotchas": ["ORACLE RULE: oracle labeler MUST be a different model lineage than
production or the regression test goes blind (green while both wrong) ? warn_if_same_lineage
encodes this but is advisory only (logs, doesn't block).", "Default labeler is _not_configured
which RAISES ? calling submit() on a default AutomatedProvider() throws NotImplementedError; you
must inject labeler via annotation_registry.get('auto', labeler=fn, lineage='...').", "submit()
is synchronous: it runs the labeler inline (despite the async submit/poll/fetch shape) and
stores results immediately; a real async oracle would resolve in poll().", "No concrete model is
wired ? this is an interface-only slice (docs/BACKLOG.md
GS-4)."]}, {"path": "backend/annotations/__init__.py", "role": "Package entry: imports file_provider
& automated_provider for their registration side-effects; re-exports registry +
ABC.", "symbols": [{"name": "__all__", "kind": "const", "sig": "['annotation_registry',
'AnnotationProvider']", "role": "Public surface."}], "contracts": ["Importing this package is what
populates annotation_registry with 'file' and 'auto'."], "edges": ["-> backend/annotations/base.py
(annotation_registry, AnnotationProvider re-export)", "-> backend/annotations/file_provider.py
(side-effect: registers FileProvider as 'file')", "-> backend/annotations/automated_provider.py
(side-effect: registers AutomatedProvider as 'auto')"], "gotchas": ["The noqa F401 imports are
load-bearing ? removing them de-registers the providers (registry would be
empty)."]}, {"path": "backend/database.py", "role": "SQLite persistence layer: videos /
play_segments / events / candidate_segments with JSON-blob extensibility, FK CASCADE, and PRAGMA
user_version migrations.", "symbols": [{"name": "Database", "kind": "class", "sig": "__init__(db_path:
str)", "role": "Owns the SQLite schema and all CRUD/query ops; inits schema on
construction."}, {"name": "Database._init_db", "kind": "func", "sig": "()", "role": "CREATE TABLE IF NOT
EXISTS for 4 tables; runs versioned migrations via PRAGMA user_version ladder (currently v1:
candidate_segments)."}, {"name": "Database.add_video", "kind": "func", "sig": "(video_id, filepath,
status, validation_results: List[Dict]) -> bool", "role": "INSERT OR REPLACE into videos;
validation_results JSON-
serialized."}, {"name": "Database.add_play_segment", "kind": "func", "sig": "(video_id, start_time,
end_time, server=None, receiver=None, metadata=None) -> Optional[int]", "role": "Insert confirmed
segment; duration computed = end-start; returns
lastrowid."}, {"name": "Database.add_event", "kind": "func", "sig": "(segment_id, timestamp,
event_type, player=None, details=None) -> bool", "role": "Insert event row (type column from
event_type); details
JSON."}, {"name": "Database.add_candidate_segment", "kind": "func", "sig": "(video_id, detector,
start_time, end_time, chunk_index=None, confidence=None, stats=None, detection_params=None) ->
Optional[int]", "role": "Persist one raw pre-confirmation detection; stats/detection_params JSON;
returns id."}, {"name": "Database.link_candidate_to_segment", "kind": "func", "sig": "(candidate_id,
segment_id) -> bool", "role": "Set candidate.confirmed_segment_id once a detection is
confirmed."}, {"name": "Database.query_candidate_segments", "kind": "func", "sig": "(video_id,
detector=None, only_unlabeled=False) -> List[Dict]", "role": "Fetch candidates (annotation/fine-
tuning workflow); deserializes stats & detection_params; ORDER BY
start_time."}, {"name": "Database.get_video", "kind": "func", "sig": "(video_id) ->
Optional[Dict]", "role": "Fetch one video; validation_results JSON-decoded (default
[])."}, {"name": "Database.query_play_segments", "kind": "func", "sig": "(video_id, filters: Dict) ->
List[Dict]", "role": "Dynamic filter query; attaches decoded metadata + nested events list per
segment."}, {"name": "Database.clear_video_data", "kind": "func", "sig": "(video_id)", "role": "DELETE
the video row (cascades to segments/events/candidates)."}], "contracts": ["videos(id TEXT PK,
filepath TEXT NOT NULL, processed_at TIMESTAMP default now, status TEXT NOT NULL,
validation_results TEXT[JSON list]).", "play_segments(id INT PK AUTOINC, video_id TEXT FK->videos
ON DELETE CASCADE, start_time REAL, end_time REAL, duration REAL, server TEXT, receiver TEXT,
metadata TEXT[JSON dict]).", "events(id INT PK, segment_id INT FK->play_segments ON DELETE
CASCADE, timestamp REAL, type TEXT, player TEXT, details TEXT[JSON
dict]).", "candidate_segments(id INT PK, video_id FK->videos CASCADE, detector TEXT NOT NULL,
chunk_index INT, start_time REAL, end_time REAL, duration REAL, confidence REAL, stats
TEXT[JSON], detection_params TEXT[JSON], confirmed_segment_id FK->play_segments ON DELETE SET
NULL, human_label TEXT, notes TEXT, created_at TIMESTAMP); index
idx_candidates_video_detector(video_id, detector).", "PRAGMA user_version is the schema contract
(currently =1); tests assert the expected version.", "query_play_segments filters keys:

```

```

duration_min(float), server(str), receiver(str), event_type(str, EXISTS on
events.type).","Returned segment dict adds 'metadata'(dict) and 'events'(list of event dicts
with decoded 'details').","Returned candidate dict has 'stats' and 'detection_params' decoded to
dicts.],"edges":["Consumes segment dicts whose start_time/end_time mirror the provider/sport
output shape (start_time/end_time floats from gemini.py / badminton.py prompt
schema).","candidate_segments table is the detector-agnostic store referenced in memory
(candidate-segments-finetuning); any new segmenter reuses it via its own
detector/stats.],"gotchas":["Every connection re-issues 'PRAGMA foreign_keys = ON' ? SQLite
disables FKs per-connection by default, so CASCADE/SET NULL only work because of this; a
connection opened elsewhere without it would silently skip cascades.","Migrations are an ordered
'if version < N' ladder ? adding a schema change requires a new block AND bumping PRAGMA
user_version, or CI version-assertion tests fail.","add_video uses INSERT OR REPLACE: re-adding
a video_id replaces the row; combined with FK CASCADE, REPLACE deletes+reinserts which can
cascade-delete dependent segments ? destructive on re-ingest.","Write helpers swallow exceptions
and return False/None (logging only) ? callers must check return values; failures are silent
otherwise.","events column is named 'type' (not 'event_type'); the method param is event_type ?
easy to confuse when writing raw SQL.","duration is stored (end-start), not derived at query
time ? inconsistent end/start with a manually-written duration would diverge.","candidate
human_label/notes columns exist but have no setter method in this file (annotation workflow
writes them elsewhere)."]]]}]

```

```

=== WASB WSL CONTRACT (external dep ~/models/WASB-SBDT, MIT; verified this session ? treat as
authoritative):
- Runs in WSL conda env 'wasb', from ~/models/WASB-SBDT/src. Hydra: compose(config_name='eval',
overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda,
runner.gpus=[0]]). gpus=[0] because this box is single-GPU.
- model=wasb -> name=hrnet, frames_in=3, frames_out=3, input 512x288 (WxH), ImageNet normalize
(mean .485/.456/.406, std .229/.224/.225); resize via affine (utils.image.get_affine_transform);
the affine matrix ('trans'/affine_mats) maps heatmap coords back to original-frame coords.
- detectors/detector.py TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis).
results[bid][eid] = list of dicts {xy: np.array([x,y]) in ORIGINAL frame coords, score: float,
scale: int}. eid indexes the frames_out frames of the window; bid is the batch element.
- trackers/online.py OnlineTracker: STATEFUL, sequential self._fid incremented per update()
(assumes every frame fed once, in order, no gaps). update(frame_dets)->{x,y,visi,score};
refresh() resets fid+track. _select_not_too_far uses cfg.tracker.max_disp; det['xy'] MUST be a
numpy array (vector math / np.linalg.norm). Re-running the tracker over the full ordered
detections is deterministic.
- dataloaders/dataset_loader.py ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train); samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}}]].
- weights pretrained_weights/wasb_badminton_best.pth.tar (NOT committed to the repo; ~6MB;
trained on academic NYCU/TrackNetV2 data -> confirm terms before COMMERCIAL deploy). monotrack
weights = noncommercial, avoided.

```

```

=== CROSS-REPO (sibling sports-data-collector, separate private repo):
- Indexer keys each video by FILE MD5 (get_file_md5 in run_process_and_stitch.py); collector
keys by human video_id. Re-encoding/re-downloading changes bytes -> MD5, so acquisition must
precede processing; the manifest tracks which exact file was used.
- Collector 'python -m src.cli.curate export' emits <out>/<sport>/<video_id>.csv in the indexer
start,end format + manifest.json -> consumed by indexer eval (backend/eval/batch.py,
run_phase3_eval.py).
- Collector also produces golden rally labels (rally_number,start_time,end_time,...) consumed by
calibrate_wasb / export_wasb_segments / rally_slicer.

```

2. assistant (2026-06-06T15:23:36.071Z)

I'll generate the CODE_MAP.md body now. The subsystem maps are comprehensive; let me verify a few load-bearing anchors before writing, then produce the document.

3. user (2026-06-06T15:23:36.565Z)

```

tests\__init__.py
tests\test_base.py
tests\test_database.py
tests\test_eval_annotations.py

```

```
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_gemini.py
tests\test_geometry.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_video.py
tests\test_yolo_hybrid.py
tests\test_annotation_providers.py
tests\test_automated_provider.py
tests\test_eval_regression.py
tests\test_rally_slicer.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_wasb_cache.py
tests\test_export_wasb_segments.py
```

4. user (2026-06-06T15:23:36.628Z)

```
scratch\test_ai_segmenter.py
scratch\test_thresholds.py
tests\test_base.py
tests\test_database.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_harness.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_gemini.py
tests\test_geometry.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_video.py
tests\test_yolo_hybrid.py
tests\test_annotation_providers.py
tests\test_automated_provider.py
tests\test_eval_regression.py
tests\test_rally_slicer.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_wasb_cache.py
tests\test_export_wasb_segments.py
```

5. assistant (2026-06-06T15:23:39.702Z)

All test files confirmed. Writing the document now.

6. user (2026-06-06T15:23:40.729Z)

```
1      """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3      Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4      detection accumulation on replay, crash-truncation recovery, manifest
5      compatibility/invalidation, and atomic trajectory writes.
```

```

6      """
7      import json
8      import os
9      import os.path as osp
10
11     import numpy as np
12
13     from backend.detectors import wasb_infer as wi
14
15
16     def _win(idx, frames):
17         """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
18         return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
19
20
21     def _write_windows(cache_dir, windows):
22         det_jsonl, _ = wi._cache_paths(cache_dir)
23         with open(det_jsonl, "a") as f:
24             wi._append_windows(f, windows)
25
26
27     # ----- #
28     # detection (de)serialization
29     # ----- #
30     def test_det_plain_and_back_roundtrip():
31         det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
32         plain = wi._det_plain(det)
33         assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
34
35         back = wi._dets_to_tracker([plain])
36         assert len(back) == 1
37         assert isinstance(back[0]["xy"], np.ndarray) # tracker does vector math on xy
38         assert back[0]["xy"].tolist() == [439.0, 64.5]
39         assert back[0]["scale"] == 0
40

```

7. user (2026-06-06T15:23:45.207Z)

```

1      import pytest
2      from backend.indexer.base import BasePlaySegmenter, SegmenterRegistry
3
4      class DummySegmenter(BasePlaySegmenter):
5          @property
6          def name(self) -> str:
7              return "dummy"
8
9          @property
10         def description(self) -> str:
11             return "Dummy segmenter for testing."
12
13         def process_video(self, video_id, video_path, player_pool=None, max_frames=None):
14             return [{"start_time": 0.0, "end_time": 1.0}]
15
16     def test_registry():
17         registry = SegmenterRegistry()
18         registry.register(DummySegmenter)
19
20         available = registry.list_available()
21         assert "dummy" in available
22         assert available["dummy"] == "Dummy segmenter for testing."
23
24         cls = registry.get("dummy")
25         assert cls == DummySegmenter

```

8. assistant (2026-06-06T15:26:29.736Z)

I have everything I need. Producing the CODE_MAP.md body.

```markdown

## CODE\_MAP.md

**\*\*Purpose:\*\*** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack.

**\*\*Regenerate:\*\*** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change.

**\*\*LEGEND\*\***

- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `\$` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

## 1. PIPELINE

### 1A. Runtime: video file -> highlight reel

```

```
config.json (json.load; absent -> {})  
  -> video_id = MD5(whole file) @backend/main.py::get_file_md5 (? chunk size varies  
per endpoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE file so  
values agree)  
  -> validate  
@backend/validators/base.py::registry.run_all(video_path, config)  
  FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> Blur -> SceneCut ->  
PTSContinuity -> Relevance  
  -> db.add_video(video_id, filepath, status, [r.dict()]) @backend/database.py  
  -> seg = segmenter_registry.get(name)(db, config) ?  
@backend/indexer/base.py::segmenter_registry  
  -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)  
  [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without  
stitching]  
  -> select segment ids -> [(start,end)] (+ stitching padding)  
  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)  
@backend/stitcher/compiler.py  
  per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->  
output/<name>.mp4  
```
```

Hybrid segmenter internal 4-phase (yolo\_hybrid / tracknet\_hybrid / wasb\_hybrid ? same shape):  
```

```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)  
  -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted via  
db.add_candidate_segment]  
  -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,  
prompt) ? provider_registry  
  -> P4 db.add_play_segment + db.link_candidate_to_segment  
```
```

Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->  
add\_play\_segment.

Legacy `motion`: pixel-diff -> segments + **\*\*fabricated\*\*** metadata/events (? not ground truth).

### 1B. WASB shuttle substrate (P2 of wasb\_hybrid)

```

```
WasbHybridSegmenter.process_video @backend/indexer/wasb_hybrid.py
```

```

-> WasbRunner.healthcheck() (gate; not ok -> return []) @backend/detectors/wasb_runner.py
-> WasbRunner.run_predict(video, out_dir)
    stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
-> @backend/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass
(CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
-> TrackNetRunner.parse_trajectory_csv(csv) @backend/detectors/tracknet_runner.py
(shared, re-exported by WasbRunner)
-> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, velocity_thresh,
merge_gap, min_window_duration) [the model-agnostic WINDOWING BRAIN]
...

(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback
(30.0, 1280.0).)

```

1C. Calibration / eval flow (NO pipeline run unless told)

```

...
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage ?
{candidates, final}
-> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
-> harness.evaluate -> EvalReport -> report_to_dict
    -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
    -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py [CI
gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate}
@backend/eval/calibration.py
Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
...
Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can
segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).

```

2. SUBSYSTEMS

2.0 Orchestration & config

```

- `@backend/main.py` ? FastAPI app + argparse CLI (`server`/`process`/`debug-
clip`/`trim`/`--setup`).
- `:::app` FastAPI; `:::db_instance`, `:::global_config` module globals set ONLY in `main()`. ?
importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPES.
- `:::process_video` `@POST /api/process` -> validate -> add_video -> segment.
`:::compile_highlights` `@POST /api/compile`. `:::run_cli_diagnose_clip` lazy `import cv2`,
samples ±3s vs `indexing.motion_threshold`.
- ? uvicorn host hardcoded `127.0.0.1`; CLI default_segmenter fallback `motion`.
- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks
automation), `default_segmenter` fallback `yolo_hybrid`, skips validation.
- `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`.
Exit 2=arg/I/O. ? pure scorer; errors if video never processed.
- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (label only).
- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok,
1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even
a regressing run).
- `@config.json` ? single config. Sections: `validation`, `indexing` (+ `tracknet`, `wasb`),
`output`, `stitching`, root `sport/ai_provider/ai_model`. ? **three default segmenters
diverge**: config ships `gemini`, setup.py writes `yolo_hybrid`, main.py fallback `motion`.
- `@setup.py` ? write default config (skips if present), ffmpeg/torch/CUDA diagnostics, GPU-
aware pip (`nvidia-smi` -> cu124 else cpu). ? generated config diverges from checked-in (no
`relevance`/`tracknet` blocks, blur 100.0 vs 20.0).

```

2.1 Segmenters (indexer) ?

- `@backend/indexer/base.py`
 - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract`
 - ``name`,`description`,`process_video`. $ `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.`
 - `::SegmenterRegistry`,`::segmenter_registry` (singleton). ? register a new segmenter: subclass BasePlaySegmenter`, call segmenter_registry.register(MyCls)` at module bottom, AND import the module in @backend/indexer/__init__.py` (registration is an IMPORT side-effect). ? register()` builds cls(None, {}) just to read .name` -> name/description MUST NOT touch self.db`/`self.config`.`
- `@backend/indexer/__init__.py`` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid``) = the de-facto registration manifest.
- `@backend/indexer/wasb_hybrid.py::WasbHybridSegmenter`` (name `'wasb_hybrid`'`) ? WSL WASB substrate. Config `indexing.wasb.velocity_threshold(0.02)`` + shared `dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers``. ? whole-video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates``; hardcoded relative out dirs `output/wasb``; needs `{PROVIDER}_API_KEY`` (missing -> []).
- `@backend/indexer/tracknet_hybrid.py::TrackNetHybridSegmenter`` (name `'tracknet_hybrid`'`) ? copy-paste twin of `wasb_hybrid`` (swaps runner; `detection_params`` adds `queue_length``). ? any P3/P4 fix must be applied to BOTH (and arguably `yolo_hybrid``).
- `@backend/indexer/yolo_hybrid.py::HybridYoloSegmenter`` (name `'yolo_hybrid`'`, no YOLO remains ? torchvision detector + supervision ByteTrack). `::_PERSON_CLASS_ID=1`` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model`,`::_extract_action_windows_from_chunk``. ? velocity computed at `fps/frame_skip`` -> `velocity_threshold`` coupled to `frame_skip``; trackers per-chunk (windows merged by TIME not ID); `import torch`` at module top.
- `@backend/indexer/gemini.py::GeminiPlaySegmenter`` (name `'gemini`'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration``; `max_workers=5`` hardcoded (ignores `ai_max_workers``); `max_segment_duration=90.0`` hardcoded; `debug_duration`` silently truncates analysis; `parse_time`` accepts colon timestamps (warns).
- `@backend/indexer/motion.py::MotionPlaySegmenter`` (name `'motion`'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint); only segmenter using `db.add_event`` and passing server/receiver; uses `print()``; honors `max_frames``.

2.2 Detectors & windowing (WSL-quarantined)

- `@backend/detectors/__init__.py`` ? docstring only (refs `docs/DECISIONS.md`` ADR-001). ? does NOT re-export runners; import submodules directly.
- `@backend/detectors/tracknet_runner.py``
 - `::TrackNetConfig`(+.from_indexing_cfg`), ::TrackNetRunner`` ? runs `TrackNet` predict.py`` in WSL.
 - `::to_wsl_mnt_path`(`C:\x`->`\mnt/c/x`), ::TrajectoryPoint`(frame,visible,x,y).`
 - **`::parse_trajectory_csv`` @staticmethod + `::trajectory_to_action_windows``**
- @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by `WasbRunner``. ? `weights_path`` defaults `''`` -> `run_predict`` returns None; `predict.py`` only `.mp4/.avi``; `visible==False`` resets prev (occlusion gap breaks track); `fps<=0->30`,`frame_width<=0->1280`` silent; `track_id_count`` hardcoded 1; `timeout_sec=0`` -> NO timeout (hangs forever).
- `@backend/detectors/wasb_runner.py::WasbRunner`` ? stages `wasb_infer.py`` + video into WSL, runs in `wasb`` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows`` = `staticmethod(TrackNetRunner.*)``. ? `run_predict`` does NOT pass cache flags (uses `wasb_infer`` defaults `flush=1000/batch=8/workers=4``); `wasb_infer.py`` MUST be re-staged each run (editing Windows copy is inert until staged); `_wsl_bash`` duplicated (drift risk).
- `@backend/detectors/wasb_infer.py`` (WSL) ? `::run`` core. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl`,`::_MANIFEST_FILE='manifest.json`,`::CACHE_VERSION=1`,`::load_and_clean_cache`` (reads longest CONTIGUOUS prefix where `w==line index``, rewrites to heal crash-truncated tail), `::manifest_compatible`,`::atomic_write_text`/_atomic_write_trajectory`,`::dets_to_tracker`` (xy MUST be np.array). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `runner.gpus=[0]`` hardcoded single-GPU; `extract_frames`` cv2-PNG SLOW (prefer host ffmpeg).

2.3 Eval & calibration (all pure; I/O lives in CLI drivers)

- `@backend/eval/metrics.py`` ? scoring primitive. `::interval_iou`,`::match_intervals`` (greedy, deterministic), `::score`->::Score`(.as_dict())` = wire shape), `::pr_curve``. ? greedy not Hungarian; `score()`` indexes the SAME preds/gts used to build `match_result`` or boundary errors are silently wrong.
- `@backend/eval/annotations.py`` ? generic GT loader. `::load_annotations`` (`start,end`` CSV; RAISES on bad rows/`start>=end``), `::parse_timestamp`` (decimal or MM:SS/HH:MM:SS), `::write_scaffold``.

```
- `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`, `::kfold_indices`. ? `generalization_gap >=0.1` = overfit alarm.
- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)` (? hardcoded dup of real defaults), `::default_grid` (45 cfgs), `::make_predict_fn`, `::load_golden_gts` (? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ? CLI defaults `fps=59.94/frame-width=1920` baked in.
- `@backend/eval/export_wasb_segments.py` ? trajectory+cfg -> golden/slicer CSV + comparison. `::TUNED=(0.05,1.0,1.0)` (? hardcoded from ONE video), `::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`. `--slice` requires `--video`.
- `@backend/eval/batch.py` ? corpus aggregation. `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path`.
- `@backend/eval/harness.py` ? DB-pred scorer. `::STAGES=('candidates','final')`, `::get_predictions` (candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})` ALL segs, detector arg ignored), `::evaluate`->`::EvalReport`->`::report_to_dict` (input contract for batch.aggregate).
- `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d), `::extract_yolo_features` (? assumes `row['stats']` already dict; raw JSON str -> all-zero), `::label_candidates` (same IoU matching as evaluator), `::build_training_set`->(X,y,intervals).
- `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn). `fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default; feature ORDER at predict must match fit (names are metadata only); fixed 1000 epochs.
- `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`, `::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`, `::regress_with_provider`. ? imports `annotation_registry` from `**backend.annotations` NOT `backend.eval.annotations` (two different "annotations"); gate fires on precision OR recall drop (F1 reported, not gated); first run on a video silently passes (no baseline).
```

2.4 Stitcher / tools / utils

```
- `@backend/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw, segments:[(start,end)], out_name)`. Per-clip re-encode -> `f concat -c copy`. $ skipped tuple `(idx,start,end,reason)`. ? if duration probe fails (0.0) NO boundary validation; `padded_start` uses `begin_padding`; temp clips in `output_dir`; raises ValueError/FileNotFoundError/RuntimeError; uses `print()`.
- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy, kept in sync manually). `_read_time` -> backend/eval/annotations.parse_timestamp.
- `@backend/utils/video.py` ? `::get_video_duration` (ffprobe), `::extract_video_chunk(...,reencode=False)`. ? uses `-t DURATION` (length, not end ts); copy mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips.
- `@backend/utils/geometry.py` ? `::get_velocity_normalised` (used by yolo_hybrid), `::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.
```

2.5 Validators ?

```
- `@backend/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic), `::ValidationRegistry`, `::registry` (singleton). $ `validate(video_path, config, context)` -> `ValidationResult`. ? **add a validator**: subclass + `registry.register(MyValidator())` (registration order = execution order). ? validators share a per-run `context` dict; exceptions wrapped as `passed=False` (never abort batch).
- `@backend/validators/format.py::FormatValidator` (`format_check`) ? **PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring `mp4`.
- `@backend/validators/resolution_fps.py::ResolutionFPSValidator` ? CONSUMES `ffprobe_meta` (? hard-fails if format_check didn't run first). `min_fps` default 29.9.
- `@backend/validators/blur.py` (`blur_check`, Laplacian var < threshold = fail),
`@../scene_cut.py` (? `cut_threshold=0.6` hardcoded, only first ~100 frames analyzed),
`@../pts_continuity.py` (? 10.0s gap hardcoded, comment says 8.0s ? drift), `@../relevance.py` (HSV court-green; lazy `import backend.sports`; ? unknown sport -> empty cfg, does NOT fail).
```

2.6 Sports / Providers / Annotations / DB ?

```
- `@backend/sports/base.py::SportProfile` ABC;
```

```
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (auto-registers badminton). ? **add a sport**:
subclass `SportProfile`, `sport_registry.register(MyProfile)`. $ prompt emits JSON array
`{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int),
shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS
timestamps; `get()` returns a fresh instance each call.
- `@backend/providers/base.py::AIVideoProvider` ABC;
`@backend/providers/gemini.py::GeminiProvider` (upload->poll->generate JSON temp 0->delete);
`@backend/providers/__init__.py::provider_registry` (auto-registers `gemini`). ? **add a
provider**: subclass, `provider_registry.register('name', MyCls)` (key passed EXPLICITLY, unlike
sport_registry). $ `analyze_video(path, prompt, chunk_duration=0, label='')` -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking `time.sleep(10)` poll; returns
`([], metrics)` on ANY failure (empty != error); lazy client (auth surfaces on first use).
- `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::STATUSES`, `::annotation_registry`. ? **add a tier**:
subclass, register at module bottom + import in `__init__.py`. $ `fetch` -> List[Interval]`
sorted; contract says RAISE (not `[]`) on unavailable (`[]` silently zeroes recall).
- `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the CSV path; ->
`backend.eval.annotations.load_annotations`).
- `@backend/annotations/automated_provider.py::AutomatedProvider` (`auto`; injectable
`labeler`, `::warn_if_same_lineage` ORACLE RULE, default `not_configured` RAISES). ? get via
`annotation_registry.get('auto', labeler=fn, lineage='...')`.
- `@backend/annotations/__init__.py` ? ? `noqa F401` imports are load-bearing (removing de-
registers providers).
- `@backend/database.py::Database` ? SQLite, 4 tables
(`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder
(currently =1). Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_segment`, `lin
k_candidate_to_segment`, `query_candidate_segments`, `query_play_segments`, `get_video`, `clear_vide
o_data`. ? every conn re-issues `PRAGMA foreign_keys=ON` (else CASCADE silently skipped);
`add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest; write
helpers swallow exceptions (return False/None); events column is `type` not `event_type`;
`duration` stored not derived.
---
```

3. DATA CONTRACTS (canonical shapes ? one place)

```
$ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
`Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
Row = `(fid, 1|0, x or -1, y or -1)`.
-> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).

$ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
`{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
`start/end = group_frame/fps + chunk_start`.

$ **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
`{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL,
confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON
{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path[,queue_length(trackn
et)],frame_skip,tracker,detector_model,detector_score_threshold(yolo)}, confirmed_segment_id,
human_label, notes, created_at}` (stats/detection_params deserialized on read).

$ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
`{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.

$ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time,
shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id`
```

stamped internally).

```
$ **GT / golden CSVs** (TWO conventions ? do not conflate):
- generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or
MM:SS/HH:MM:SS; RAISES on bad row.
- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.

$ **det_raw.jsonl** (one window/line): `{ "w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...] }`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
$ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
$ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv}, ...]}`; ?
DB key = file MD5, manifest `video_id` is a label.

$ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches,unmatched_pred(=FP),unmatched_gt(=FN)}`.
`Score{iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

$ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]), 'score':float,
'scale':int }`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST rebuild `xy` as
np.array (tracker does `np.linalg.norm`).
```

4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; verified ? authoritative)

```
- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval',
overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda,
runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU.
- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet
normalize (mean .485/.456/.406, std .229/.224/.225); affine resize
(`utils.image.get_affine_transform`); affine matrix maps heatmap coords -> original-frame
coords.
- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. `eid`
= frames_out index, `bid` = batch element.
- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential `self._fid++` per `update()`
(assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`;
`refresh()` resets. `_select_not_too_far` uses `cfg.tracker.max_disp`; `det['xy']` MUST be
np.array. Full ordered re-run is deterministic.
- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform,
seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]}]`.
- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed (~6MB), trained on
academic NYCU/TrackNetV2 data -> ? confirm terms before COMMERCIAL deploy. monotrack weights =
noncommercial, avoided.
```

5. CROSS-REPO (sibling private repo `sports-data-collector`)

```
- Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-
downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact
file used.
- `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer
`start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed
by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
```

6. GOTCHAS (cross-cutting footguns)

- ****MD5 keying:**** ``video_id`` = MD5 of WHOLE file everywhere ? but each entrypoint uses a different chunk SIZE (64KB/1MB/8KB); values agree, code does not. Manifest ``video_id`` is a label, NOT the DB key.
- ****Three default segmenters:**** `config.json`gemini`` vs `setup.py`yolo_hybrid`` vs `main.py`motion``. You may run a different detector than expected.
- ****Gemini free-tier / API-key dead path:**** `hybrids + gemini`` return ``[]`` (not error) if ``{PROVIDER}_API_KEY`` missing OR WSL healthcheck fails ? a silent no-op for callers. ``analyze_video`` also returns ``[]`` on real failure -> empty != "no rallies".
- ****fps / frame_width must be PROBED not assumed:**** ``_probe_fps_width`` / runner fallbacks (30.0, 1280.0/1920.0) silently mis-scale velocity thresholds if probing is skipped; `calibrate_wasb`CLI`` bakes ``59.94/1920``.
- ****WSL`/mnt` can't see Google Drive / network mounts;**** stage video into native WSL fs (``~/clips``) ? ``/mnt/c`` reads are slow and Drive-backed paths invisible.
- ****ffmpeg NOT in WSL**** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. ``extract_frames`` uses cv2 PNG (slow) ? prefer host ffmpeg + ``--frames_dir``.
- ****GPU float nondeterminism:**** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
- ****`output/` is gitignored:**** DB, baselines, cache, reels live there. Predictions MUST exist in ``output/<db_name>`` before any eval/regression.
- ****Validator ordering coupling:**** ``resolution_fps_check`` hard-fails unless ``format_check`` ran earlier (shared ``context['ffprobe_meta']``).
- ****`add\_video` INSERT OR REPLACE + FK CASCADE**** -> re-ingesting a ``video_id`` cascade-deletes its segments. Use ``clear_video_data`` deliberately.
- ****Twin segmenters:**** ``wasb_hybrid.py`` / ``tracknet_hybrid.py`` are copy-paste; fixes to P3/P4 must land in both.
- ****Two "annotations" modules:**** ``backend.eval.annotations`` (GT CSV loader) vs ``backend.annotations`` (provider registry). `regression.py`` imports the latter.
- ****Hardcoded tuned constants:**** ``calibrate_wasb.BASELINE``, ``export.TUNED``, `slicer/compiler`padding`` ? duplicated by copy, not import; re-tuning won't propagate.
- ****`timeout\_sec=0` = no timeout:**** a hung WSL/GPU call blocks forever.
- ****Stitch concat assumes identical encode:**** per-clip re-encode then ``-c copy`` concat; if probe fails (0.0) no boundary validation.

7. RAMP PATHS (to do X -> open Y::symbol)

```
| Task | Open |
|---|---|
| Tune candidate windowing (velocity/merge/min-dur) |
`@backend/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json`
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
| Add a segmenter | subclass `@backend/indexer/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in `@backend/indexer/__init__.py` |
| Change stitching padding | `config.json` stitching.{begin_padding,end_padding}`; impl
`@backend/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
| Add a sport | subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)`
in `@backend/sports/__init__.py`; define prompt+HSV in new file |
| Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', CIs)` |
| Add an annotation/GT tier | subclass `@backend/annotations/base.py::AnnotationProvider`;
register + import in `@backend/annotations/__init__.py` |
| Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory`
--labels --fps --frame-width`); trust `cv` recall not `fit` |
| Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main` (`--slice`
needs `--video`) |
| Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
| Score DB preds vs GT (one video) | `@run_eval.py::main` |
| Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
| Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress, 2=missing
```

```

DB; snapshot via `--update-baseline` |
| Resume a crashed WASB pass | nothing to do ? `@backend/detectors/wasb_infer.py::run` auto-
resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart; cache
invalidates on weights/sport/frames change |
| Add a validator | subclass `@backend/validators/base.py::VideoValidator`;
`registry.register(instance)` (mind order; producers before consumers) |
| Add a DB column/table | `@backend/database.py::Database._init_db` migration ladder + bump
`PRAGMA user_version` (else version-assert tests fail) |
| Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |

---

```

8. TEST MAP (`tests/test\_\*.py` -> subsystem)

```

| Test | Covers |
|---|---|
| `test_base.py` | `indexer.base.SegmenterRegistry` (register/get/list via DummySegmenter) |
| `test_yolo_hybrid.py` / `test_tracknet_hybrid.py` | hybrid segmenters (yolo
torchvision+ByteTrack; tracknet 4-phase) |
| `test_gemini.py` | `indexer.gemini.GeminiPlaySegmenter` (parse/validate/chunk) |
| `test_tracknet_runner.py` | `detectors.tracknet_runner` parse_trajectory_csv +
trajectory_to_action_windows (shared brain) |
| `test_wasb_cache.py` | `detectors.wasb_infer` resumable cache (serialize, contiguity heal,
manifest invalidation, atomic writes) |
| `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
| `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate |
| `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict |
| `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate |
| `test_calibration.py` | `eval.calibration` kfold/select_best/cross_validate/LOVO |
| `test_calibrate_wasb.py` | `eval.calibrate_wasb` CLI/run/grid |
| `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment+comparison export |
| `test_eval_training.py` | `eval.training` feature extraction + labeling |
| `test_eval_classifier.py` | `eval.classifier.LogisticRegression` fit/predict/save/load |
| `test_eval_regression.py` | `eval.regression` baseline compare/gate |
| `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label parse |
| `test_video.py` | `utils.video` duration/chunk |
| `test_geometry.py` | `utils.geometry` center/distance/velocity/IoU |
| `test_database.py` | `database.Database` schema/CRUD/migrations/version |
| `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile |
| `test_providers.py` / `test_provider_gemini.py` / `test_gemini.py` | `providers` registry +
GeminiProvider lifecycle |
| `test_annotation_providers.py` / `test_automated_provider.py` | `annotations` registry,
FileProvider, AutomatedProvider (oracle/lineage) |
| (no dedicated test) | `validators/*`, `stitcher.compiler`, `main.py`, `run_*.py` orchestrators
?? exercised only indirectly |
...

```